DATA STRUCTURES & ALGORITHMS

23CSE203

LAB MANUAL

Submitted By	
Name	Y.Nipun
Roll No.	AV.SC.U4CSE25054
Year/Sem/Section	2nd year /Sem-3/CSE-A
Credits	05

Submitted To	
Name	Dr.Rajkumar Batchu(S.I Grade)
Designation	Asst. Professor
Department	School Of Computing

Index

S.No.	Title	Date	Page No.
1)	**Week-1:** **1A)** Rocket launch systems display the countdown of before launch instead of using a loop implement the countdown using the recursion write a recursion Python program to display the countdown from n to 1 followed by launch. **1B)** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive Python program to calculate P^n (power), where P = Principal growth factor, n = number of years. **1C)** An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **1D)** In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula. Develop a recursive program to calculate the factorial of a given number. **1E)** Write a recursive Python program to print the first n terms of a Fibonacci series.	23-07-26	5-13
2)	**Week-2:** **2A)** Write a python program to perform linear search on the given input. **2B)** Write a python program to perform binary search on the given input.Verify by taking sorted array and unsorted array as input.	30-07-26	14-17

S.No.	Title	Date	Page No.
3)	**Week-3:** **3A)** Write a python program to sort the elements using a selection sort technique.Take an input elements from the user. **3B)** Write a python program to sort the elements using a insertion sort technique.Take an input elements from the user. **3C)** Write a python program to sort the elements using a bubble sort technique.Take an input elements from the user. **3D)** Write a python program to sort the elements using a merge sort technique. Take an input elements from the user. **3E)** Write a python program to sort the elements using a quick sort technique.Take an input elements from the user.	06-08-26	18-27
4)	**Week-4:** **4A)** Write a Python program to implement singly linked list which could perform the below operations by taking the input from the user: **4B)** Write a Python program to implement doubly linked list which could perform the below operations by taking the input from the user: **4C)** Write a Python program to implement circular linked list which could perform the below operations by taking the input from the user:	13-08-26	28-48

S.No.	Title	Date	Page No.
5)	**Week-5:** **5A)** Write a python program to implement a stack using an Array, Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.	27-08-26	49-55
6)	**Week-6:** **6A)** Write a python program to implement a queue using an Array, Linked list, circular queue. Implement the queue operations such as enqueue, dequeue, peak and also display the queue elements. **6B)**Write a python program to implement a circular queue using an Array, Linked list.	03-09-26	56-73
7)	**Week-7:** **7A)** Write a python program to implement binary search tree using an array and linked list with all its operations.	24-09-26	74-88
8)	**Week-8:** **8A)** Write a python program to implement Binary tree with its operations using array and linked list.	08-10-26	89-95
9)	**Week-9:** **9A)** Write a python code for AVL tree using array and linked list with all its operations.	22-10-26	96-117
10)	**Week-10:** **10A)** Write a Python program to implement hashing using separate chaining, linear probing, and quadratic probing.	29-10-26	118-119

WEEK-1

1A)Aim: Rocket launch systems display the countdown of before launch instead of using a loop implement the countdown using the recursion write a recursion Python program to display the countdown from n to 1 followed by launch.

Program:

1a.py - D:\DSA-25054\Week-1\1a.py (3.13.7)

File Edit Format Run Options Window Help

```
def countdown(n):
    if n == 0:
        print("Launch")
    else:
        print(n)
        countdown(n - 1)

n = int(input("Enter countdown number:"))

countdown(n)
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: D:\DSA-25054\Week-1\1a.py ==========
Enter countdown number:4
4
3
2
1
Launch
```

Explanation:

- def defines the countdown() function.
- input() accepts the countdown value from the user.
- int() converts the input into an integer.
- countdown() starts the recursive countdown process.
- if checks the base case (n == 0).
- else executes when the condition is False.
- print(n) displays the current countdown number.

- countdown(n - 1) recursively calls the function with the value decreased by 1.
- print("Launch!") displays the launch message when the countdown reaches 0.
- The program demonstrates the use of functions, recursion, conditional statements, input, and output to perform the countdown.

Error Table:

S.No.	Errors Observed	Rectification
1)	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
2)	Missing base case (if n == 0) causing infinite recursion.	Include the base case to stop recursion .l
3)	incorrect indentation leading to an Indentation Error	Maintain proper Python indentation.
4)	Missing colon (:) after def, if, or else.	Add the required colon (:) after statements.
5)	Calling countdown(n) instead of countdown(n - 1), causing infinite recursion.	Decrease the value by 1 in each recursive call using countdown(n - 1).

1B)Aim: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive Python program to calculate P^n (power), where P = Principal growth factor, n = number of years.

Program:

1b.py - D:\DSA-25054\Week-1\1b.py (3.13.7)

File Edit Format Run Options Window Help

```
def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)

p = int(input("Enter principal:"))
n = int(input("Enter number of years:"))

result = power(p, n)
print("Power =", result)
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: D:\DSA-25054\Week-1\1b.py ==========
Enter principal:100
Enter number of years:2
Power = 10000
```

Explanation:

- power() is the recursive function that computes P raised to n.
- The base case is n == 0, which returns 1 (any number to the power 0 is 1).
- The recursive case multiplies p with power(p, n - 1), reducing n by 1 on each call.
- float() allows the principal growth factor to be a decimal value.
- input() collects the values of P and n from the user.
- Each recursive call is stacked until the base case is reached, then the results are multiplied back up the call stack.
- print() displays the final computed value of P^n.

Error Table:

S.No.	Errors Observed	Rectification
1)	Missing base case (n == 0) causing infinite recursion.	Include the base case to stop recursion.
2)	Entering a negative value for n causes the recursion to never reach 0.	Restrict input to non-negative integers or handle negative n separately.
3)	Using int() for P truncates decimal growth factors.	Use float() to accept decimal values for P.
4)	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5)	Calling power(p, n) instead of power(p, n - 1), causing infinite recursion.	Decrease n by 1 in each recursive call using power(p, n - 1).

1C) Aim: An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

Program:

1c.py - D:\DSA-25054\Week-1\1c.py (3.13.7)

File Edit Format Run Options Window Help

```
def search_employee(emp_list, emp_id, index=0):
    if index == len(emp_list):
        return -1
    elif emp_list[index] == emp_id:
        return index
    else:
        return search_employee(emp_list, emp_id, index + 1)

employee_ids = [101, 102, 103, 104, 105]
search_id = int(input("Enter Employee ID to search: "))

position = search_employee(employee_ids, search_id)

if position != -1:
    print("Employee ID", search_id, "found at position", position)
else:
    print("Employee ID", search_id, "not found")
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: D:\DSA-25054\Week-1\1c.py ==========
Enter Employee ID to search: 101
Employee ID 101 found at position 0
>>>
```

Explanation:

- search_employee() is the recursive function that searches the list one index at a time.
- The base case index == len(emp_list) means every element was checked and the ID was not found, so it returns -1.
- The second condition checks if the current element matches the employee ID being searched.
- If not matched, the function calls itself with index + 1, moving to the next employee in the list.
- employee_ids is the list of employee IDs stored by the HR department.
- input() and int() collect the employee ID to search for from the user.
- The if-else block reports whether the employee ID was found and at which position.

Error Table:

S.No.	Errors Observed	Rectification
1)	Missing base case (index == len(emp_list)) causing infinite recursion.	Include the base case to stop recursion when the list is exhausted.
2)	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
3)	Calling search_employee(emp_list, emp_id, index) without incrementing index, causing infinite recursion.	Increase index by 1 in each recursive call using index + 1.
4)	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5)	Searching an empty list without handling it, giving a confusing result.	Add a check for an empty list before searching.

1D) Aim: In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula. Develop a recursive program to calculate the factorial of a given number.

Program:

1d.py - D:\DSA-25054\Week-1\1d.py (3.13.7)

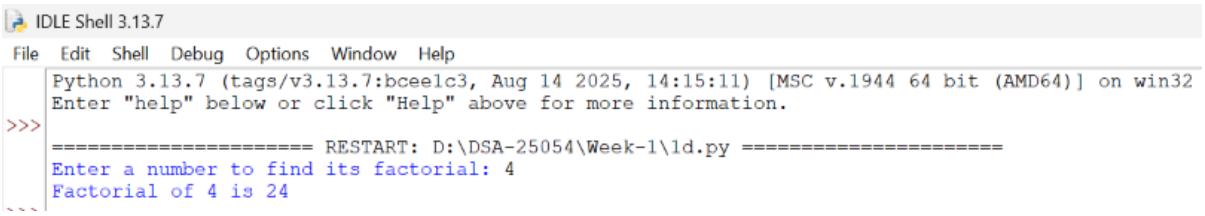
File Edit Format Run Options Window Help

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

n = int(input("Enter a number to find its factorial: "))

print("Factorial of", n, "is", factorial(n))
```

Output:



Explanation:

- factorial() is the recursive function that computes n! (n factorial).
- The base case is n == 0 or n == 1, both of which return 1.
- The recursive case multiplies n with factorial(n - 1), reducing n by 1 on each call.
- input() and int() collect the number whose factorial is to be found.
- Each recursive call is stacked until the base case is reached, then the results are multiplied back up the call stack.

Error Table:

S.No.	Errors Observed	Rectification
1)	Missing base case (n == 0 or n == 1) causing infinite recursion.	Include the base case to stop recursion.
2)	Entering a negative number, for which factorial is undefined.	Restrict input to non-negative integers.
3)	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
4)	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5)	Calling factorial(n) instead of factorial(n - 1), causing infinite recursion.	Decrease n by 1 in each recursive call using factorial(n - 1).

1E) Aim: Write a recursive Python program to print the first n terms of a Fibonacci series.

Program:

1e.py - D:\DSA-25054\Week-1\1e.py (3.13.7)

File Edit Format Run Options Window Help

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)
n = int(input("Enter the number of terms: "))

print("Fibonacci series:")
for i in range(n):
    print(fibonacci(i), end=" ")
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: D:\DSA-25054\Week-1\1e.py ==========
Enter the number of terms: 2
Fibonacci series:
0 1
```

Explanation:

- fibonacci() is the recursive function that returns the nth Fibonacci term.
- The base cases are n == 0 (returns 0) and n == 1 (returns 1).
- The recursive case adds fibonacci(n - 1) and fibonacci(n - 2), following the Fibonacci rule.
- input() and int() collect the number of terms to be printed.
- The for loop calls fibonacci(i) for each index from 0 to n - 1.
- print(..., end=" ") displays each term on the same line, separated by a space.

Error Table:

S.No.	Errors Observed	Rectification
1)	Missing base cases (n == 0 and n == 1) causing	Include both base cases to

	infinite recursion.	stop recursion.
2)	Entering a negative number of terms.	Restrict input to non-negative integers.
3)	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.
4)	Incorrect indentation leading to an IndentationError.	Maintain proper Python indentation.
5)	Using fibonacci(n - 1) - fibonacci(n - 2) instead of addition, giving wrong terms.	Use addition, fibonacci(n - 1) + fibonacci(n - 2), as per the Fibonacci rule.

WEEK-2

2A) Aim: Write a python program to perform linear search on the given input.

Program:

2a.py - D:\DSA-25054\Week-2\2a.py (3.13.7)

File Edit Format Run Options Window Help

```
def linear_search(arr,target):
    n = len(arr)
    for i in range (n):
        if (arr[i] == target):
            return i
    return -1

n = int(input("Enter the number of elements:"))
arr = []
print("Enter the elements:")

for i in range(n):
    arr.append(int(input()))
target = int(input("Enter the element to be found:"))
result = linear_search(arr,target)

if(result != -1):
    print(f"Element {target} found at {result} index")
else:
    print("Element not found in the given array")
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: D:\DSA-25054\Week-2\2a.py ==========
Enter the number of elements:2
Enter the elements:
1
2
Enter the element to be found:1
Element 1 found at 0 index
```

Explanation:

- linear_search(arr, target) sequentially iterates through arr to check each element against target.
- for i in range(n): iterates through array indices from 0 to n-1 sequentially.

- if arr[i] == target: checks for a match and returns index i if found.
- return -1 executes when the target element is absent from the array.
- arr.append(int(input())) converts user input to integers and builds the array.
- if (result != -1): displays the index where the element is found or prints a "not found" message.

Error Table:

S.No.	Errors Observed	Rectification
1)	Invalid (non-numeric) input entered by the user.	Entered a valid integer as input.
2)	Incorrect indentation leading to an Indentation Error	Maintained proper Python indentation.
3)	Missing colon (:) after def, for, if, or else.	Added the required colon (:) after statements.
4)	ValueError when passing non-integer strings to int(input()).	Ensure user input consists of valid integer characters.
5)	IndexError caused by accessing an array index out of bounds.	Ensure loop bounds do not exceed len(arr) - 1.

2B) Aim: Write a python program to perform binary search on the given input.Verify by taking sorted array and unsorted array as input.

Program:

2b.py - D:\DSA-25054\Week-2\2b.py (3.13.7)

File Edit Format Run Options Window Help

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1


n = int(input("Enter the number of elements: "))
print("Enter the elements:")
arr = []
for i in range(n):
    arr.append(int(input()))

target = int(input("Enter the element to be found: "))

if arr != sorted(arr):
    print("Array is not sorted")
    arr.sort()
    print(f"Sorted array: {arr}")

result = binary_search(arr, target)

if result != -1:
    print(f"Element {target} found at index {result} in the sorted array")
else:
    print("Element not found in the array")
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: D:\DSA-25054\Week-2\2b.py ==========
Enter the number of elements: 2
Enter the elements:
10
1
Enter the element to be found: 1
Array is not sorted
Sorted array: [1, 10]
Element 1 found at index 0 in the sorted array
```

Explanation:

- binary_search(arr, target) uses a divide-and-conquer strategy on a sorted array.
- while low <= high: runs as long as the search boundary contains valid elements.
- mid = (low + high) // 2 computes the middle index using integer division.
- if arr[mid] == target: returns mid if the target is found at the middle index.
- elif arr[mid] < target: updates low = mid + 1, otherwise sets high = mid - 1.
- arr.sort() ensures the input array is sorted before executing binary search.

Error Table:

S.No.	Errors Observed	Rectification
1)	Performing binary search on an unsorted array leading to incorrect results.	Sort the array using arr.sort() before applying binary search.
2)	Using floating-point division / instead of integer division // for calculating mid, causing a TypeError.	Use mid = (low + high) // 2 for integer index calculation.
3)	Incorrect loop condition (low < high instead of low <= high), missing target when low == high.	Change the condition to while low <= high:.
4)	Incorrectly updating pointers like low = mid or high = mid causing an infinite loop.	Use low = mid + 1 and high = mid - 1.
5)	Invalid (non-numeric) input entered by the user.	Enter a valid integer as input.

WEEK-3

3A) Aim: Write a python program to sort the elements using a selection sort technique.Take an input elements from the user.

Program:

selectionsort.py - C:/Users/Nipun Yellapu/OneDrive/Pictures/Documents/selectionsort.py (3.13.7)

File Edit Format Run Options Window Help

```
n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    value = int(input(f"Enter element {i + 1}: "))
    arr.append(value)

for i in range(n - 1):

    min_index = i

    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j

    arr[i], arr[min_index] = arr[min_index], arr[i]

print("Sorted array:", arr)
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Nipun Yellapu/OneDrive/Pictures/Documents/selectionsort.py =
Enter number of elements: 5
Enter element 1: 2
Enter element 2: 4
Enter element 3: 1
Enter element 4: 8
Enter element 5: 9
Sorted array: [1, 2, 4, 8, 9]
```

Explanation:

Took the number of elements from the user.

- Created an empty list arr.
- Used a for loop to take each element individually.
- min_index stores the position of the smallest element.
- The inner loop searches for the smallest element in the unsorted portion.
- Swapped the smallest element with the first unsorted element.
- Repeated this until the array was sorted

Error Table:

S.No.	Errors Observed	Rectification
1)	Incorrect initialization of min_index, causing the minimum element to be identified incorrectly.	Initialize min_index = i before searching for the minimum element.
2)	Starting the inner loop from 0, causing already sorted elements to be checked again.	Start the inner loop from i + 1.
3)	Forgetting to update min_index when a smaller element is found.	Use min_index = j inside the if condition.
4)	Incorrect swapping of elements, resulting in an unsorted array.	Use arr[i], arr[min_index] = arr[min_index], arr[i].
5)	Invalid (non-numeric) input entered by the user.	Enter a valid integer for each array element.
6)	Incorrect indentation causing an IndentationError.	Maintain proper indentation inside for loops and if statements.

3B) Aim: Write a python program to sort the elements using a insertion sort technique.Take an input elements from the user.

Program:

insertionsort.py - C:/Users/Nipun Yellapu/OneDrive/Pictures/Documents/insertionsort.py (3.13.7)

File Edit Format Run Options Window Help

```
n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    arr.append(int(input(f"Enter element {i + 1}: ")))


for i in range(1, n):

    key = arr[i]
    j = i - 1

    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1

    arr[j + 1] = key

print("Sorted array:", arr)
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Nipun Yellapu/OneDrive/Pictures/Documents/insertionsort.py =
Enter number of elements: 5
Enter element 1: 1
Enter element 2: 9
Enter element 3: 2
Enter element 4: 4
Enter element 5: 8
Sorted array: [1, 2, 4, 8, 9]
```

Explanation:

- Took the number of elements from the user.
- Used a loop to input every element.
- Started sorting from index 1.

- Stored the current element in key.
- Compared key with the elements before it.
- Shifted larger elements one position to the right.
- Inserted key into its correct position.
- Repeated until the array was sorted.

Error Table:

S.No.	Errors Observed	Rectification
1)	Starting the outer loop from 0, unnecessarily processing the first element.	Start the loop from 1 because the first element is already considered sorted.
2)	Forgetting to store the current element in key.	Use key = arr[i] before shifting elements.
3)	Forgetting j -= 1, causing the while loop to run indefinitely.	Decrease j by using j -= 1 after shifting an element.
4)	Placing the key at arr[j] instead of arr[j + 1].	Insert the key using arr[j + 1] = key.
5)	Incorrect condition in the while loop.	Use j >= 0 and arr[j] > key to shift larger elements.

3C) Aim: Write a python program to sort the elements using a bubble sort technique.Take an input elements from the user.

Program:

bubblesort.py - C:/Users/Nipun Yellapu/OneDrive/Pictures/Documents/bubblesort.py (3.13.7)

File Edit Format Run Options Window Help

```
n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    arr.append(int(input(f"Enter element {i + 1}: ")))

for i in range(n - 1):
    for j in range(n - 1 - i):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

print("Sorted array:", arr)
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/Nipun Yellapu/OneDrive/Pictures/Documents/bubblesort.py ==
Enter number of elements: 5
Enter element 1: 1
Enter element 2: 8
Enter element 3: 9
Enter element 4: 2
Enter element 5: 4
Sorted array: [1, 2, 4, 8, 9]
```

Explanation:

- Took the number of elements.
- Input each element using a loop.
- Compared two adjacent elements.
- If the left element was greater, we swapped them.
- After every pass, the largest remaining element moved to the end.

Error Table:

S.No.	Errors Observed	Rectification
1)	Comparing non-adjacent elements instead of neighboring elements.	Compare arr[j] with arr[j + 1].
2)	Using an incorrect inner loop range.	Use range(n - 1 - i) so that already sorted elements are not checked
3)	Forgetting to swap elements when they are in the wrong order.	Use arr[j], arr[j + 1] = arr[j + 1], arr[j].
4)	Using < instead of > for ascending order.	Use if arr[j] >arr[j + 1] for ascending sorting.
5)	Using an incorrect outer loop range.	Use range(n - 1) to perform the required passes.
6)	Entering non-numeric values as array elements.	Enter valid integer values using int(input()).

3D) Aim: Write a python program to sort the elements using a merge sort technique. Take an input elements from the user.

Program:

mergesort.py - C:/Users/Nipun Yellapu/OneDrive/Pictures/Documents/mergesort.py (3.13.7)

File Edit Format Run Options Window Help

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1

        else:
            result.append(right[j])
            j += 1

    while i < len(left):
        result.append(left[i])
        i += 1

    while j < len(right):
        result.append(right[j])
        j += 1

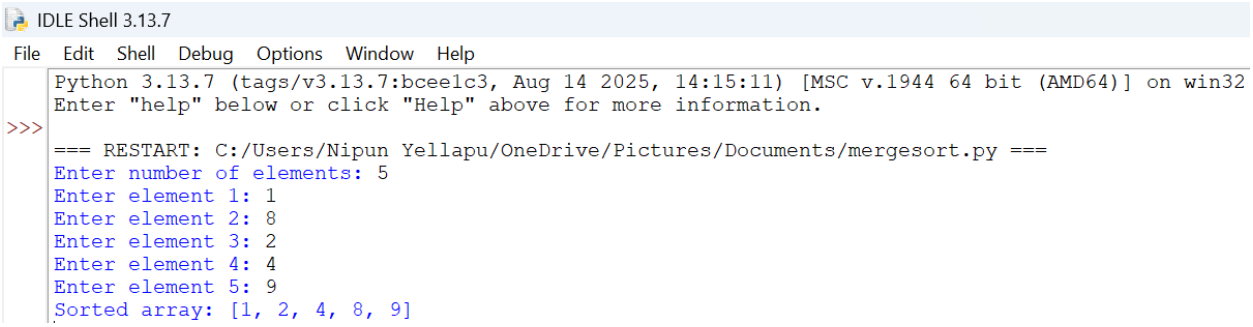
    return result

n = int(input("Enter number of elements: "))
arr = []

for i in range(n):
    arr.append(int(input(f"Enter element {i + 1}: ")))

arr = merge_sort(arr)
print("Sorted array:", arr)
```

Output:



Explanation:

- Took the number of elements from the user.
- Used a loop to input the array.
- Used recursion to divide the array.
- Divided the array into two halves.
- Continued dividing until each part contained one element.
- Used the merge() function to combine the smaller arrays.
- Compared elements from the left and right halves.
- Added the smaller element to result.
- Added remaining elements.
- Returned the sorted array.

Error Table:

S.No.	Errors Observed	Rectification
1)	Missing base case causing infinite recursion.	Include if len(arr) <= 1: return arr to stop recursion.
2)	Incorrect calculation of the middle position.	Use mid = len(arr) // 2 to divide the array correctly.
3)	Incorrectly dividing the array into left and right parts.	Use arr[:mid] for the left half and arr[mid:] for the right half.
4)	Forgetting to increment i or	Increment the corresponding index after

	j while merging.	adding an element to result.
5)	Forgetting to add the remaining elements from either half.	Add the remaining elements using result.extend(left[i:]) and result.extend(right[j:]).

3E) Aim: Write a python program to sort the elements using a quick sort technique.Take an input elements from the user.

Program:

quicksort.py - C:/Users/Nipun Yellapu/OneDrive/Pictures/Documents/quicksort.py (3.13.7)

File Edit Format Run Options Window Help

```
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1

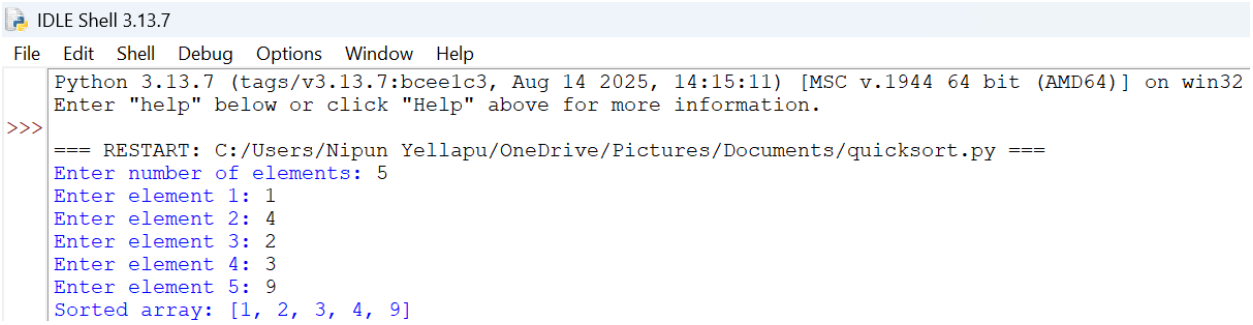
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quick_sort(arr, low, high):
    if low < high:
        pivot_index = partition(arr, low, high)
        quick_sort(arr, low, pivot_index - 1)
        quick_sort(arr, pivot_index + 1, high)

n = int(input("Enter number of elements: "))
arr = []

for i in range(n):
    arr.append(int(input(f"Enter element {i + 1}: ")))
quick_sort(arr, 0, n - 1)
print("Sorted array:", arr)
```

Output:



Explanation:

- Took the number of elements.
- Used a loop to input every element.
- Selected the last element as the pivot.
- Compared every element with the pivot.
- Elements smaller than or equal to the pivot were moved to the left.
- Elements greater than the pivot remained on the right.
- Placed the pivot in its correct position.
- Recursively sorted the left and right portions.
- Continued until the entire array was sorted.

Error Table:

S.No.	Errors Observed	Rectification
1)	Missing base condition causing infinite recursion.	Use if low < high: to continue recursion only when elements remain to be sorted.
2)	Incorrectly selecting or accessing the pivot element.	Select the last element using pivot = arr[high].
3)	Forgetting to return the pivot index from the partition function.	Use returni + 1 after placing the pivot.
4)	Incorrect recursive boundaries causing the pivot to be processed again.	Use quick_sort(arr, low, pivot_index - 1) and quick_sort(arr, pivot_index + 1, high).

WEEK-4

4A)Aim: Write a Python program to implement singly linked list which could perform the below operations by taking the input from the user:

1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at a specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

Program:

4a.py - D:/DSA-25054/4a.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    # 1. Create a linked list
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter value: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head
                while temp.next is not None:
                    temp = temp.next
                temp.next = new_node
```

```
    # 2. Insert at beginning
    def insert_beginning(self):
        data = int(input("Enter value: "))
        new_node = Node(data)

        new_node.next = self.head
        self.head = new_node

    # 3. Insert at end
    def insert_end(self):
        data = int(input("Enter value: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        temp = self.head
        while temp.next is not None:
            temp = temp.next

        temp.next = new_node
```

```
    # 4. Insert at a specific index
    def insert_index(self):
        data = int(input("Enter value: "))
        index = int(input("Enter index: "))

        new_node = Node(data)

        if index == 0:
            new_node.next = self.head
            self.head = new_node
            return

        temp = self.head
        prev = None
        count = 0

        while temp is not None and count < index:
            prev = temp
            temp = temp.next
            count += 1

        if count != index:
            print("Invalid index")
            return

        prev.next = new_node
        new_node.next = temp
```

```
    # 5. Delete at a specific index
    def delete_index(self):
        index = int(input("Enter index to delete: "))

        if self.head is None:
            print("List is empty")
            return

        if index == 0:
            self.head = self.head.next
            return

        temp = self.head
        prev = None
        count = 0

        while temp is not None and count < index:
            prev = temp
            temp = temp.next
            count += 1

        if temp is None:
            print("Invalid index")
            return

        prev.next = temp.next
```

```
    # 6. Delete first node
    def delete_first(self):
        if self.head is None:
            print("List is empty")
            return

        self.head = self.head.next

    # 7. Delete last node
    def delete_last(self):
        if self.head is None:
            print("List is empty")
            return

        if self.head.next is None:
            self.head = None
            return

        temp = self.head

        while temp.next.next is not None:
            temp = temp.next

        temp.next = None

    # 8. Count number of nodes
    def count_nodes(self):
        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)
```

```
    # 9. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")


# Main program
linked_list = LinkedList()

while True:
    print("\n----- SINGLY LINKED LIST -----")
    print("1. Create a linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at a specific index")
    print("5. Delete at a specific index")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        linked_list.create()

    elif choice == 2:
        linked_list.insert_beginning()
```

```
    elif choice == 3:
        linked_list.insert_end()

    elif choice == 4:
        linked_list.insert_index()

    elif choice == 5:
        linked_list.delete_index()

    elif choice == 6:
        linked_list.delete_first()

    elif choice == 7:
        linked_list.delete_last()

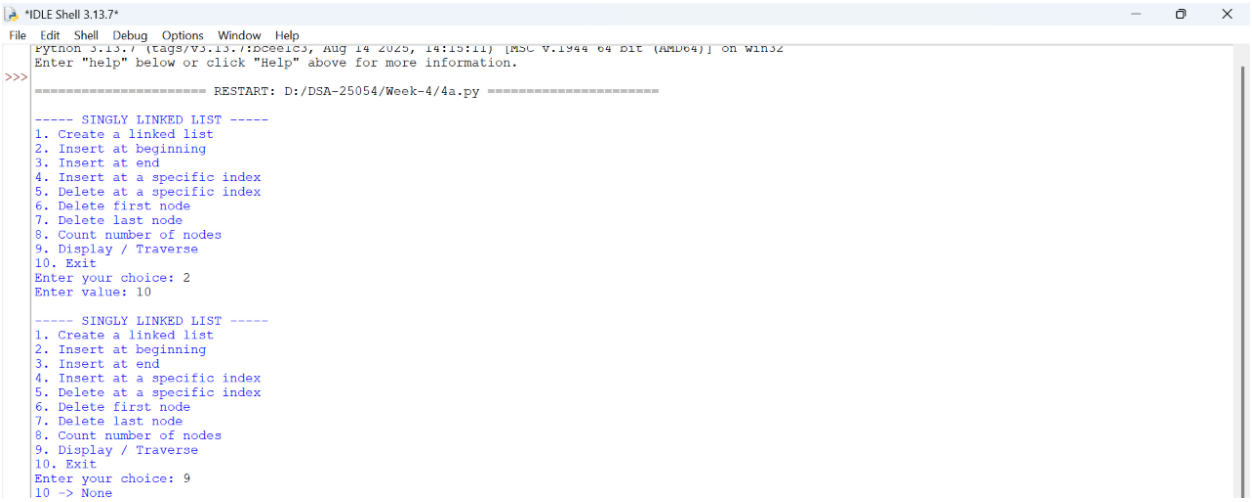
    elif choice == 8:
        linked_list.count_nodes()

    elif choice == 9:
        linked_list.display()

    elif choice == 10:
        print("Exiting...")
        break

    else:
        print("Invalid choice")
```

Output:



Explanation:

1. Node Class:

   • Created a Node to store data and the next node's address.

   • Initialized next as None.

2. LinkedList Class:

   • Created the linked list using the head pointer.

   • Initially, head is set to None.

3. Create a Linked List:

   • Took the number of nodes and their values from the user.

   • Created and connected each node to the list.

4. Insert at Beginning:

   • Created a new node with the given value.

   • Connected it to the current head and made it the new head.

5. Insert at End:

   • Created a new node.

   • Traversed to the last node and connected the new node.

6. Insert at Specific Index:

   • Took the value and 0-based index from the user.

   • Traversed to the required position and inserted the new node.

7. Delete by Value:

   • Searched for the given value in the list.

   • Changed the links to remove the matching node.

8. Delete First Node:

- Moved the head to the next node.
- This removes the first node.

9. Delete Last Node:
   - Traversed to the second-last node.
   - Set its next pointer to None.
10. Count Nodes:
   - Traversed through the list.
   - Increased the counter for each node and displayed the count.
11. Display / Traverse:
   - Started from the head and visited every node.
   - Printed each node's data until None was reached.
12. Main Program:
   - Displayed a menu for all operations.
   - Executed the selected operation until the user chose Exit.

Error Table:

S.No.	Errors Observed	Rectification
1)	Trying to insert or delete when the linked list is empty.	Check if head is None before performing the operation.
2)	Incorrectly updating the head while inserting at the beginning.	Set the new node as the head and connect it to the old head.
3)	Forgetting to update the link while inserting at the end.	Traverse to the last node and set its next pointer to the new node

4B)Aim: Write a Python program to implement doubly linked list which could perform the below operations by taking the input from the user:

1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at a specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

Program:

4b.py - D:/DSA-25054/Week-4/4b.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter value: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next

                temp.next = new_node
                new_node.prev = temp

    def insert_beginning(self):
        data = int(input("Enter value: "))
        new_node = Node(data)

        if self.head is not None:
            new_node.next = self.head
            self.head.prev = new_node

        self.head = new_node
```

4b.py - D:/DSA-25054/Week-4/4b.py (3.13.7)

File Edit Format Run Options Window Help

```
            self.head = new_node

        def insert_end(self):
            data = int(input("Enter value: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
                return

            temp = self.head
            while temp.next:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp

        def insert_index(self):
            data = int(input("Enter value: "))
            index = int(input("Enter index: "))

            if index == 0:
                new_node = Node(data)

                if self.head:
                    new_node.next = self.head
                    self.head.prev = new_node

                self.head = new_node
                return

            temp = self.head

            for i in range(index - 1):
                if temp is None:
                    print("Invalid index")
                    return
                temp = temp.next

            if temp is None:
                print("Invalid index")
                return
```

4b.py - D:/DSA-25054/Week-4/4b.py (3.13.7)

File Edit Format Run Options Window Help

```
        new_node.next = temp.next
        new_node.prev = temp

        if temp.next:
            temp.next.prev = new_node

        temp.next = new_node

    def delete_value(self):
        value = int(input("Enter value to delete: "))

        temp = self.head

        while temp and temp.data != value:
            temp = temp.next

        if temp is None:
            print("Value not found")
            return

        if temp.prev:
            temp.prev.next = temp.next
        else:
            self.head = temp.next

        if temp.next:
            temp.next.prev = temp.prev

        print("Node deleted")

    def delete_first(self):
        if self.head is None:
            print("List is empty")
            return

        self.head = self.head.next

        if self.head:
            self.head.prev = None

    def delete_last(self):
        if self.head is None:
            print("List is empty")
            return
```

4b.py - D:/DSA-25054/Week-4/4b.py (3.13.7)

File Edit Format Run Options Window Help

```
        temp = self.head

        while temp.next:
            temp = temp.next

        if temp.prev:
            temp.prev.next = None
        else:
            self.head = None

    def count(self):
        count = 0
        temp = self.head

        while temp:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)

    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")
```

4b.py - D:/DSA-25054/Week-4/4b.py (3.13.7)

File Edit Format Run Options Window Help

```
            temp = temp.next

        print("None")

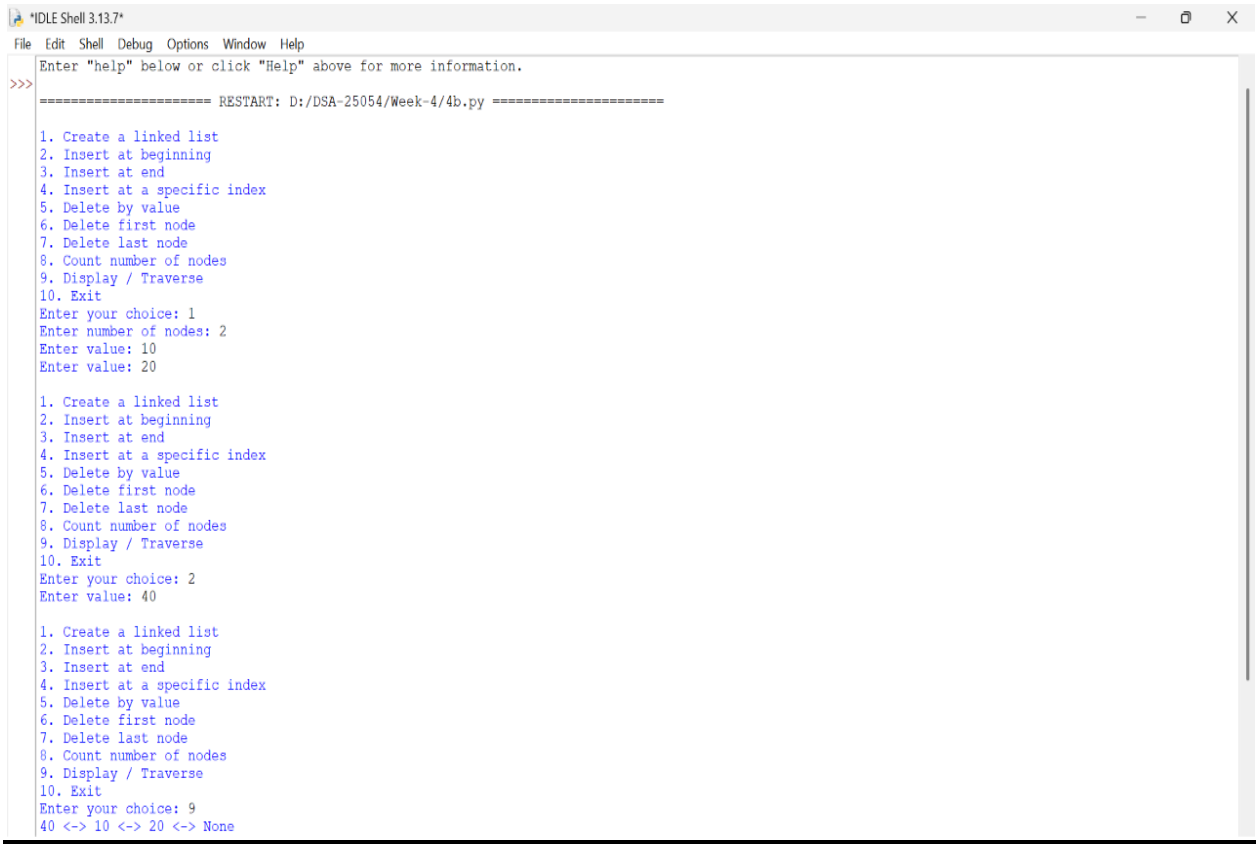

dll = DoublyLinkedList()

while True:
    print("\n1. Create a linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at a specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()
    elif choice == 2:
        dll.insert_beginning()
    elif choice == 3:
        dll.insert_end()
    elif choice == 4:
        dll.insert_index()
    elif choice == 5:
        dll.delete_value()
    elif choice == 6:
        dll.delete_first()
    elif choice == 7:
        dll.delete_last()
    elif choice == 8:
        dll.count()
    elif choice == 9:
        dll.display()
    elif choice == 10:
        print("Exiting...")
        break
    else:
        print("Invalid choice")
```

Output:



Explanation:

1. Node Class:
   • Created a Node to store data, previous node address, and next node address.

2. DoublyLinkedList Class:
   • Created the linked list using the head pointer, which is initially set to None.

3. Create a Linked List:
   • Took the number of nodes and values from the user and connected the nodes using prev and next pointers.

4. Insertion Operations:
   • Inserted nodes at the beginning, end, and at a specific index by updating the required prev and next links.

5. Delete by Value:
   • Searched for the given value and removed the matching node by changing the links of neighboring nodes.

6. Delete First and Last Node:
   • Updated the head for the first node and changed the previous node's next pointer when deleting the last node.

7. Count Nodes:
   • Traversed the list using the next pointer and increased the counter for every node.

8. Display / Main Program:
   • Displayed all nodes using traversal and provided a menu to perform all operations until Exit.

Error Table:

S.No.	Errors Observed	Rectification
1)	Forgetting to update the prev pointer while inserting a node.	Update both prev and next pointers.
2)	Incorrectly updating the head while inserting at the beginning.	Make the new node the head and update the old head's prev pointer.
3)	Deleting a node without updating both links.	Update the prev and next links of neighboring nodes.
4)	Trying to delete from an empty list.	Check if head is None before deletion.
5)	Invalid index entered during insertion.	Check the index and display an invalid index message.

4C)Aim: Write a Python program to implement circular linked list which could perform the below operations by taking the input from the user:

1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at a specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

Program:

5a.py - D:/DSA-25054/5a.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter value: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
                new_node.next = self.head
            else:
                temp = self.head

                while temp.next != self.head:
                    temp = temp.next

                temp.next = new_node
                new_node.next = self.head
```

5a.py - D:/DSA-25054/5a.py (3.13.7)

File Edit Format Run Options Window Help

```
                temp.next = new_node
                new_node.next = self.head

    def insert_beginning(self):
        data = int(input("Enter value: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
            return

        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        new_node.next = self.head
        temp.next = new_node
        self.head = new_node

    def insert_end(self):
        data = int(input("Enter value: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
            return

        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        temp.next = new_node
        new_node.next = self.head
```

5a.py - D:/DSA-25054/5a.py (3.13.7)

File Edit Format Run Options Window Help

```
        temp.next = new_node
        new_node.next = self.head

    def insert_index(self):
        data = int(input("Enter value: "))
        index = int(input("Enter index: "))

        if index == 0:
            self.insert_beginning()
            return

        if self.head is None:
            print("Invalid index")
            return

        temp = self.head

        for i in range(index - 1):
            temp = temp.next

            if temp == self.head:
                print("Invalid index")
                return

        new_node = Node(data)
        new_node.next = temp.next
        temp.next = new_node
```

5a.py - D:/DSA-25054/5a.py (3.13.7)

File Edit Format Run Options Window Help

```
        new_node = Node(data)
        new_node.next = temp.next
        temp.next = new_node

    def delete_value(self):
        value = int(input("Enter value to delete: "))

        if self.head is None:
            print("List is empty")
            return

        current = self.head
        previous = None

        while True:
            if current.data == value:
                if current == self.head:
                    if current.next == self.head:
                        self.head = None
                    else:
                        last = self.head

                        while last.next != self.head:
                            last = last.next

                        self.head = self.head.next
                        last.next = self.head
                else:
                    previous.next = current.next

                print("Node deleted")
                return

            previous = current
            current = current.next

            if current == self.head:
                break

        print("Value not found")
```

5a.py - D:/DSA-25054/5a.py (3.13.7)

File Edit Format Run Options Window Help

```
    def delete_first(self):
        if self.head is None:
            print("List is empty")
            return

        if self.head.next == self.head:
            self.head = None
            return

        last = self.head

        while last.next != self.head:
            last = last.next

        self.head = self.head.next
        last.next = self.head

    def delete_last(self):
        if self.head is None:
            print("List is empty")
            return

        if self.head.next == self.head:
            self.head = None
            return

        temp = self.head

        while temp.next.next != self.head:
            temp = temp.next

        temp.next = self.head
```

5a.py - D:/DSA-25054/5a.py (3.13.7)

File Edit Format Run Options Window Help

```
            temp = temp.next

        temp.next = self.head

    def count(self):
        if self.head is None:
            print("Number of nodes: 0")
            return

        count = 0
        temp = self.head

        while True:
            count += 1
            temp = temp.next

            if temp == self.head:
                break

        print("Number of nodes:", count)

    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next

            if temp == self.head:
                break

        print("(Head)")
```

5a.py - D:/DSA-25054/5a.py (3.13.7)

File Edit Format Run Options Window Help

```
                break

            print("(Head)")

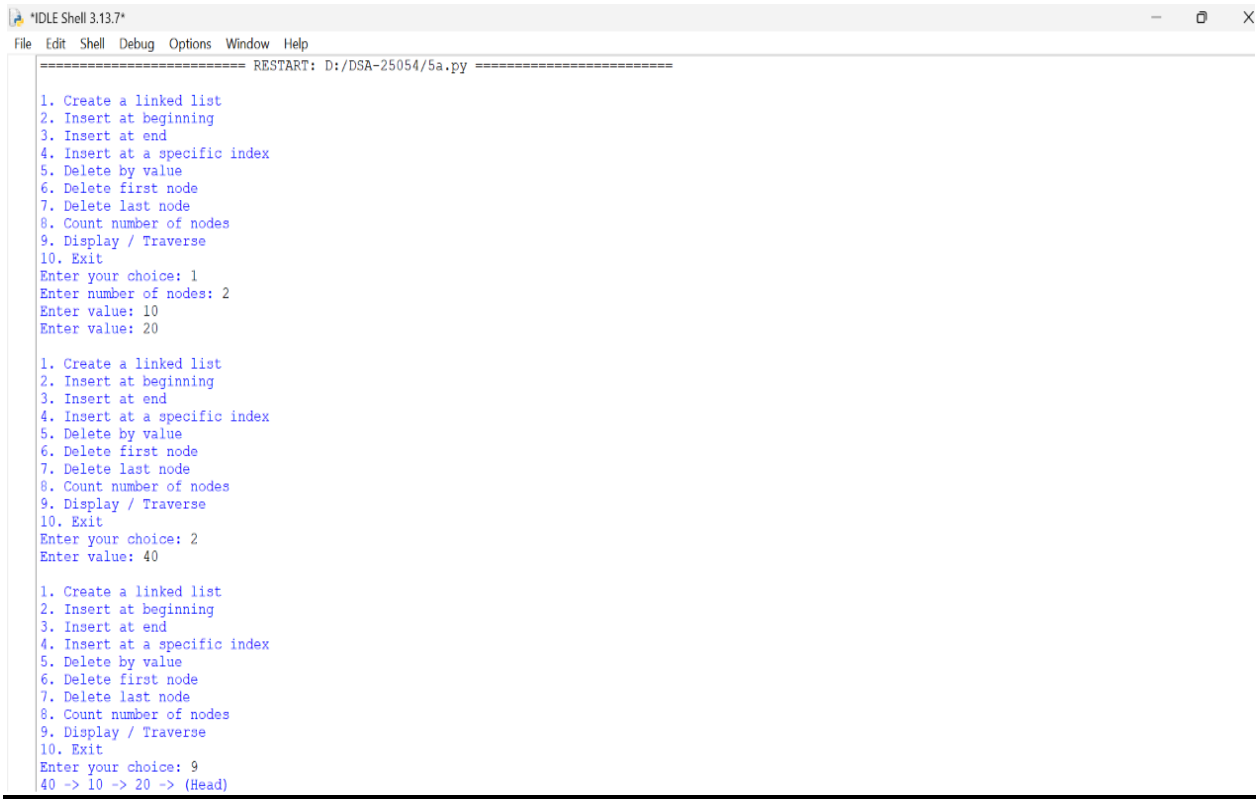

cll = CircularLinkedList()

while True:
    print("\n1. Create a linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at a specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        cll.create()
    elif choice == 2:
        cll.insert_beginning()
    elif choice == 3:
        cll.insert_end()
    elif choice == 4:
        cll.insert_index()
    elif choice == 5:
        cll.delete_value()
    elif choice == 6:
        cll.delete_first()
    elif choice == 7:
        cll.delete_last()
    elif choice == 8:
        cll.count()
    elif choice == 9:
        cll.display()
    elif choice == 10:
        print("Exiting...")
        breakD
    else:
        print("Invalid choice")
```

Output:



Explanation:

1. Node Class:
   • Created a Node to store data and the next node's address.

2. CircularLinkedList Class:
   • Created the circular linked list using the head pointer, initially set to None.

3. Create a Linked List:
   • Took the number of nodes and values from the user and connected the last node back to the head.

4. Insertion Operations:
   • Inserted nodes at the beginning, end, and at a specific index while maintaining the circular connection.

5. Delete by Value:
   • Searched for the given value and removed the matching node by updating the required next pointer.

6. Delete First and Last Node:
   • Updated the head or the last node's next pointer while maintaining the circular structure.

7. Count Nodes:
   • Traversed the list and counted nodes until the traversal reached the head again.

8. Display / Main Program:
   • Displayed all nodes and provided a menu to perform the operations until Exit.

Error Table:

S.No.	Errors Observed	Rectification
1)	Forgetting to connect the last node back to the head.	Set the last node's next pointer to head.
2)	Using None as the stopping condition during traversal.	Stop traversal when the current node reaches head again.
3)	Infinite loop during traversal.	Check if the current node is equal to head.
4)	Incorrectly updating the head while inserting at the beginning.	Update head and connect the last node to the new head.
5)	Trying to delete from an empty list.	Check if head is None before deletion.

WEEK 5

5A)Aim: Write a python program to implement a stack using an Array, Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.

Stack using Array:

5a Arrays.py - D:\DSA-25054\Week-5\5a Arrays.py (3.13.7)

File Edit Format Run Options Window Help

```
class StackArray:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into the stack")

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item, "popped from the stack")

    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Stack elements (top to bottom):")
            for i in range(self.top, -1, -1):
                print(self.stack[i])
```

```
size = int(input("Enter size of stack: "))
s = StackArray(size)

while True:
    print("\n--- STACK USING ARRAY ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Exiting program...")
        break
    else:
        print("Invalid choice!. Try again.")
```

Output:




```
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (top to bottom):
20

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting program...
```

\

Stack Using Linked List:

5a Linked List.py - D:\DSA-25054\Week-5\5a Linked List.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class StackLinkedList:
    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print(data, "pushed into the stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            popped = self.top.data
            self.top = self.top.next
            print(popped, "popped from the stack")

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Stack elements (top to bottom):")
            temp = self.top
            while temp:
                print(temp.data)
                temp = temp.next
```

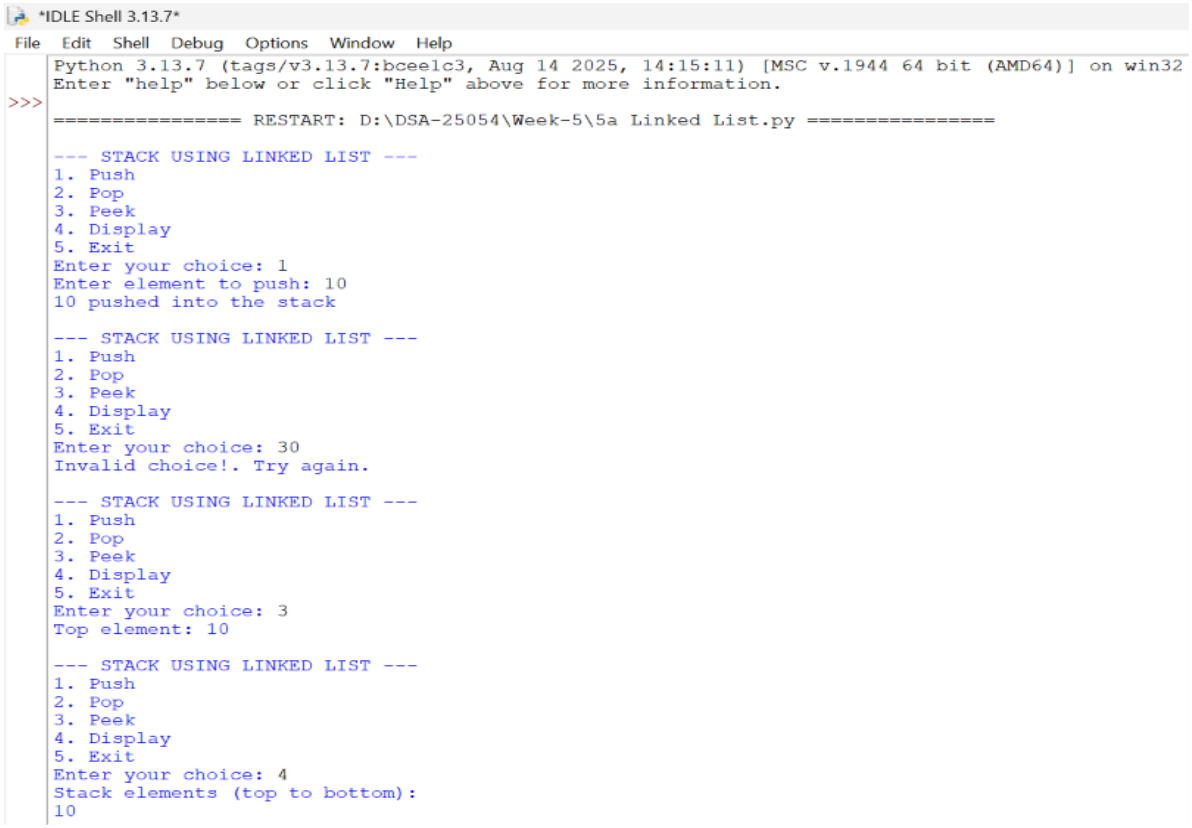
```
sll = StackLinkedList()

while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter element to push: "))
        sll.push(item)
    elif choice == 2:
        sll.pop()
    elif choice == 3:
        sll.peek()
    elif choice == 4:
        sll.display()
    elif choice == 5:
        print("Exiting program...")
        break
    else:
        print("Invalid choice!. Try again.")
```

Output:



```
*IDLE Shell 3.13.7*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
================ RESTART: D:\DSA-25054\Week-5\5a Linked List.py ================

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 10
10 pushed into the stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 30
Invalid choice!. Try again.

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 10

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (top to bottom):
10
```

Explanation:

Stack Using Array

Structure:

- Uses a Python list (self.stack) to store elements.
- self.top keeps track of the index of the last pushed element.
- self.size defines the maximum capacity.

Operations:

1. Push
    - Check if top == size - 1 → Overflow.
    - Otherwise, increment top and insert the element at that position.
2. Pop
    - Check if top == -1 → Underflow (empty stack).
    - Otherwise, remove the element at top, decrement top.
3. Peek
    - Shows the element at top without removing it.
4. Display
    - Prints elements from top down to 0 (top to bottom order)

Stack Using Linked List

Structure:

- Each element is a Node with:
    - data (value)
    - next (pointer to the next node).
- self.top points to the most recently pushed node.

Operations:

1. Push
    - Create a new node.
    - Link its next to the current top.
    - Update top to this new node.
2. Pop
    - If top is None → Underflow.
    - Otherwise, remove the node at top and update top to top.next.
3. Peek
    - Shows the data at top.
4. Display
    - Traverse from top downwards, printing each node's data.

Error Table:

S.No.	Errors Observed	Rectification
1.	def _init_(self, size):	Python expects **two underscores** before and after init.
2.	Display on empty stack.	Add condition: if self.top is None: → print "Stack is empty".
3.	Stack Overflow when pushing beyond the defined size	Add a check: if self.top == self.size - 1: → print "Stack Overflow".
4.	Stack Underflow when popping from empty stack	Add a check: if self.top is None: → print "Stack Underflow".

WEEK-6

6A)**Aim :**Write a python program to implement a queue using an Array, Linked list, circular queue. Implement the queue operations such as enqueue, dequeue, peak and also display the queue elements.

Program: Using Arrays

6a Arrays.py - D:\DSA-25054\Week-6\6a Arrays.py (3.13.7)

File Edit Format Run Options Window Help

```
class Queue:
    def __init__(self, size=5):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
            self.rear += 1
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            self.front += 1
            print(f"{x} deleted from the queue")

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    def peek(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])

size = int(input("Enter the size of Queue: "))
q = Queue(size)

while True:
    print("\n----- Queue Menu -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Exiting Program")
        break
    else:
        print("Invalid choice")
```

Output:

```
*IDLE Shell 3.13.7*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: D:\DSA-25054\Week-6\6a Arrays.py ==========
Enter the size of Queue: 2

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 10
10 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 20
20 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 30
Queue Overflow
```

```
IDLE Shell 3.13.7
File  Edit  Shell  Debug  Options  Window  Help
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 30
Queue Overflow

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
10 deleted from the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 20

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the queue are:
20

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting Program
```

Explanation:

Queue Using Array

Structure:

- Uses a Python list self.queue to store elements.
- self.front keeps track of the **first element** in the queue.
- self.rear keeps track of the **last element** in the queue.
- self.size defines the **maximum capacity** of the queue.
- Initially:
    - front = -1
    - rear = -1

Operations:

1. Enqueue

- Check if rear == size - 1 → **Overflow** (queue is full).
- If the queue is empty, set front = 0.
- Increment rear.
- Insert the element at rear.

2. Dequeue

- Check if front == -1 or front > rear → **Underflow** (queue is empty).
- Remove the element at front.
- Increment front.
- If the queue becomes empty, reset front and rear to -1.

3. Peek

- Shows the element at front without removing it.
- If the queue is empty → Underflow/empty message.

4. Display

- Prints elements from front to rear.
- Elements are displayed in **front-to-rear order**.

Important Points:

- Queue follows **FIFO (First In, First Out)**.
- Insertion takes place at the **rear**.
- Deletion takes place at the **front**.
- **Overflow** → queue is full.
- **Underflow** → queue is empty.

Error Table:

S.No.	Errors Observed	Rectification
1.	Display on empty circular queue.	Add condition: if self.front == -1: → print "Queue is empty".
2.	Circular Queue Overflow	Add a check: if (self.rear + 1) % self.size == self.front: → print "Circular Queue Overflow".
3.	Circular Queue Underflow	Add a check: if self.front == -1: → print "Circular Queue Underflow".
4.	rear does not wrap around to the beginning of the array.	Update using: self.rear = (self.rear + 1) % self.size.

Program: Using Linked List

6a Linked List.py - D:\DSA-25054\Week-6\6a Linked List.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, value):
        new_node = Node(value)

        if self.rear is None:
            self.front = new_node
            self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

        print(value, "inserted")

    def dequeue(self):
        if self.front is None:
            print("Queue is Empty")
        else:
            print(self.front.data, "deleted")
            self.front = self.front.next

            if self.front is None:
                self.rear = None

    def peek(self):
        if self.front is None:
            print("Queue is Empty")
        else:
            print("Front element:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is Empty")
        else:
            temp = self.front
```

```
q = Queue()

while True:
    print("\n--- QUEUE MENU ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        q.enqueue(value)

    elif choice == 2:
        q.dequeue()

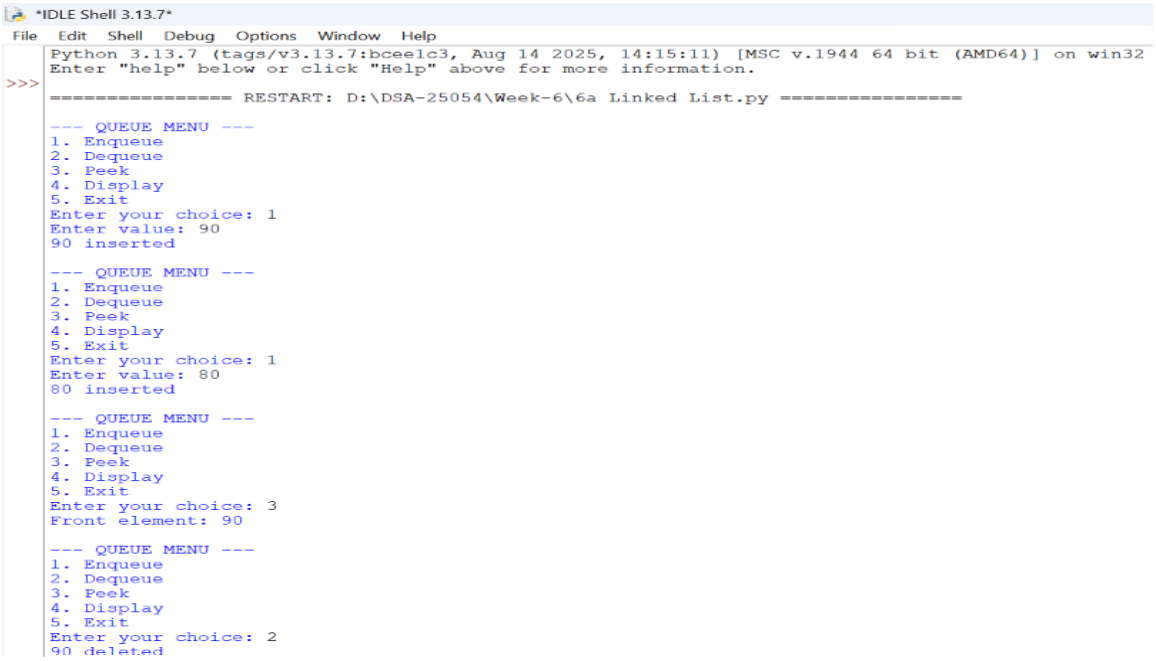
    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Program Ended")
        break

    else:
        print("Invalid choice")
```

Output:



```
--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: 80

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice:
```

Explanation:

Queue Using Linked List

Structure:

- Each element is represented using a **Node**.
- Each Node contains:
  - data → stores the value.
  - next → points to the next node.
- self.front points to the **first node**.
- self.rear points to the **last node**.
- Initially:
  - front = None
  - rear = None

Operations:

1. Enqueue

- Create a new node.
- If the queue is empty:
  - Set both front and rear to the new node.
- Otherwise:

- Link rear.next to the new node.
- Update rear to the new node.

2. Dequeue

- Check if front is None → **Underflow**.
- Store/remove the node at front.
- Move front to front.next.
- If front becomes None, set rear = None.

3. Peek

- Shows the data stored at front without removing it.
- If front is None, the queue is empty.

4. Display

- Start from front.
- Traverse using next.
- Print each element until None.

Important Points:

- Follows **FIFO**.
- Insertion → **rear**.
- Deletion → **front**.
- No fixed size is required.
- Memory is allocated dynamically.

Error Table: Queue Using Linked List

S.NO	Error Observed	Rectification
1	Displaying the queue when it is empty.	Check if self.front is None: and print "Queue is empty".
2	Deleting an element from an empty queue.	Check whether self.front is None and print "Queue Underflow".
3	front and rear not initialized for the first node.	Set self.front = self.rear = new when the queue is empty.
4	New node not linked to the existing queue.	Use self.rear.next = new to link the new node.
5	rear not updated after insertion.	Update self.rear = new after inserting the node.
6	front not updated after deletion.	Update self.front = self.front.next after deleting the first node.
7	rear not reset after deleting the last node.	Set self.rear = None when self.front becomes None.
8	Incorrect traversal while displaying the queue.	Update the temporary pointer using temp = temp.next.

6B)Aim : Write a python program to implement a circular queue using an Array, Linked list.

Program: Using Arrays

6b Arrays.py - D:\DSA-25054\Week-6\6b Arrays.py (3.13.7)

File Edit Format Run Options Window Help

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        elif self.front == -1:
            self.front = 0
            self.rear = 0
            self.queue[self.rear] = item
            print(item, "inserted into the queue")
        else:
            self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = item
            print(item, "inserted into the queue")

    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        elif self.front == self.rear:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = -1
            self.rear = -1
            print(item, "deleted from the queue")
        else:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = (self.front + 1) % self.size
            print(item, "deleted from the queue")

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])
```

6b Arrays.py - D:\DSA-25054\Week-6\6b Arrays.py (3.13.7)

File Edit Format Run Options Window Help

```
    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            i = self.front
            while True:
                print(self.queue[i])
                if i == self.rear:
                    break
                i = (i + 1) % self.size

size = int(input("Enter the size of the queue: "))
q = CircularQueue(size)

while True:
    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: D:\DSA-25054\Week-6\6b Arrays.py ==========
Enter the size of the queue: 2

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 20
20 inserted into the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 98
98 inserted into the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 20

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
20 deleted from the queue
```

```
----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the queue are:
98

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

Explanation:

Circular Queue Using Array

Structure:

- Uses a Python list self.queue to store elements.
- self.front points to the **first element**.
- self.rear points to the **last element**.
- self.size defines the maximum capacity.
- When rear reaches the last index, it can **wrap around to index 0**.
- This allows previously freed positions to be reused.

Operations:

1. Enqueue

- Check if (rear + 1) % size == front → **Overflow** (circular queue is full).
- If the queue is empty:
    - Set front = 0
    - Set rear = 0
- Otherwise:

- Move rear using:
  rear = (rear + 1) % size

- Insert the element at rear.

2. Dequeue

- Check if front == -1 → **Underflow**.
- Remove the element at front.
- If front == rear:
  - The queue becomes empty.
  - Set front = -1 and rear = -1.
- Otherwise:
  - Move front using:
    front = (front + 1) % size

3. Peek

- Shows the element at front without removing it.
- If front == -1, the queue is empty.

4. Display

- Start from front.
- Move towards rear.
- Use modulo % size to wrap around to the beginning of the array.

Important Points:

- Follows **FIFO**.
- Insertion → **rear**.
- Deletion → **front**.
- The last position connects logically back to the first position.
- **Unused spaces can be reused**, unlike a simple linear array queue.

- Overflow occurs when the next rear position becomes the front.

Error Table: Circular Queue Using Array

S.No.	Errors Observed	Rectification
1.	Display on empty queue.	Add condition: if self.front == -1: → print "Queue is empty".
2.	Queue Overflow when inserting beyond the defined size.	Add a check: if self.rear == self.size - 1: → print "Queue Overflow".
3.	Queue Underflow when deleting from an empty queue.	Add a check: if self.front == -1 or self.front>self.rear: → print "Queue Underflow".
4.	First element inserted without setting front.	If the queue is empty, set self.front = 0 before increasing rear.

Program: Using Linked List

6b Linked List.py - D:\DSA-25054\Week-6\6b Linked List.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularQueueLinkedList:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)
        if self.front is None:
            self.front = new_node
            self.rear = new_node
            self.rear.next = self.front
        else:
            self.rear.next = new_node
            self.rear = new_node
            self.rear.next = self.front
        print(f"{data} inserted into circular queue")

    def dequeue(self):
        if self.front is None:
            print("Circular Queue Underflow")
            return

        if self.front == self.rear:
            item = self.front.data
            self.front = None
            self.rear = None
        else:
            item = self.front.data
            self.front = self.front.next
            self.rear.next = self.front

        print(f"{item} deleted from circular queue")

    def peek(self):
        if self.front is None:
            print("Circular Queue is empty")
        else:
            print("Front element:", self.front.data)
```

```
    def display(self):
        if self.front is None:
            print("Circular Queue is empty")
            return

        print("Circular Queue elements are:")
        temp = self.front
        while True:
            print(temp.data)
            temp = temp.next
            if temp == self.front:
                break

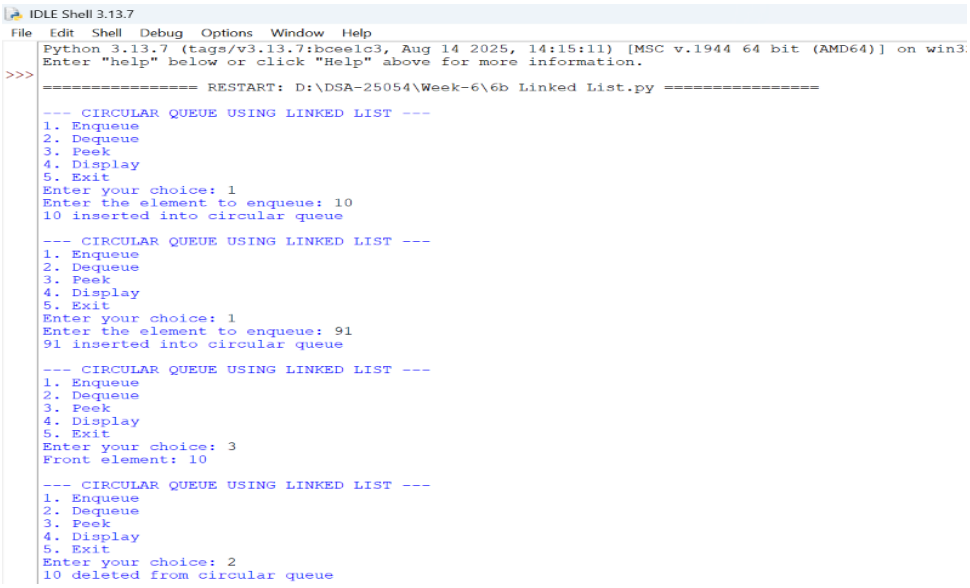
cq_ll = CircularQueueLinkedList()

while True:
    print("\n--- CIRCULAR QUEUE USING LINKED LIST ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        cq_ll.enqueue(item)
    elif choice == 2:
        cq_ll.dequeue()
    elif choice == 3:
        cq_ll.peek()
    elif choice == 4:
        cq_ll.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

Output:




```
--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Circular Queue elements are:
91

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

Circular Queue Using Linked List

Structure:

- Each element is represented using an individual Node.
- Each node contains two fields:
  - data stores the element value.
  - next points to the next node in the circular chain.
- self.front points to the first node of the queue.
- self.rear points to the last node of the queue.

- Unlike a linear linked list queue where self.rear.next = None, in a circular linked list queue, self.rear.next permanently points back to self.front.
- Initially:
    - front = None
    - rear = None

Operations:

1. **Enqueue:**

    - Create a new node with the given data.
    - If the queue is empty (self.front is None):

        - Set self.front = new_node.
        - Set self.rear = new_node.
        - Connect the rear node back to the front: self.rear.next = self.front.

    Otherwise:

        - Link the current rear node to the new node: self.rear.next = new_node.
        - Update rear to the new node: self.rear = new_node.
        - Connect the updated rear back to the front: self.rear.next = self.front.

2. **Dequeue:**

    - Check if self.front is None $\rightarrow$ Underflow (queue is empty).
    - If self.front == self.rear (only one node in the queue):

        - Retrieve the data.
        - Set self.front = None and self.rear = None to mark the queue empty.

    Otherwise:

        - Retrieve the data from self.front.
        - Advance self.front to the next node: self.front = self.front.next.
        - Maintain circularity by pointing rear to the updated front: self.rear.next = self.front.

3. **Peek:**

    - Shows the data stored at self.front without removing it.
    - If self.front is None, display that the queue is empty.

4. **Display:**

    - Check if self.front is None; if so, print "Queue is empty".
    - Set a temporary traversal pointer: temp = self.front.
    - Repeatedly print temp.data and move temp = temp.next.
    - Terminate the loop when temp == self.front to avoid an infinite loop.

Important Points:

- Strictly follows FIFO (First In, First Out).
- Insertion takes place at the rear.
- Deletion takes place at the front.
- No fixed size limit is required; memory is allocated dynamically as nodes are added.
- Queue overflow does not occur under normal execution because nodes are created on demand.
- The last node's next reference always points back to self.front to maintain an unbroken ring.

Error Table: Circular Queue Using Linked List

S.No.	Errors Observed	Rectification
1.	Display on empty queue.	Add condition: if self.front == -1: → print "Queue is empty".
3.	Queue Underflow when deleting from an empty queue.	Add a check: if self.front == -1 or self.front>self.rear: → print "Queue Underflow".

WEEK-7

7A)**Aim:**Write a python program to implement binary search tree using an Array and Linked List with all its operations.

Program: Using Array

7a.py - D:/DSA-25054/Week-7/7a.py (3.13.7)

File Edit Format Run Options Window Help

```
class BST:
    def __init__(self):
        self.tree = [None] * 100

    def insert(self, data):
        i = 0
        while True:
            if self.tree[i] is None:
                self.tree[i] = data
                return
            if data < self.tree[i]:
                i = 2 * i + 1
            else:
                i = 2 * i + 2

    def search(self, data):
        i = 0
        while i < 100 and self.tree[i] is not None:
            if self.tree[i] == data:
                return True
            if data < self.tree[i]:
                i = 2 * i + 1
            else:
                i = 2 * i + 2
        return False

    def min(self, i):
        temp = self.tree[i]
        while temp:
            if self.tree[2 * i + 1] is None:
                break
            i = 2 * i + 1
            temp = self.tree[i]
        return temp
```

7a.py - D:/DSA-25054/Week-7/7a.py (3.13.7)

File Edit Format Run Options Window Help

```
    def delete(self, data):
        i = 0
        while i < 100 and self.tree[i] is not None:
            if self.tree[i] == data:
                # No child
                if self.tree[2 * i + 1] is None and self.tree[2 * i + 2] is None:
                    self.tree[i] = None
                    return True
                # Only right child
                elif self.tree[2 * i + 1] is None:
                    self.tree[i] = self.tree[2 * i + 2]
                    self.tree[2 * i + 2] = None
                    return True
                # Only left child
                elif self.tree[2 * i + 2] is None:
                    self.tree[i] = self.tree[2 * i + 1]
                    self.tree[2 * i + 1] = None
                    return True
                # Two children
                else:
                    temp = self.min(2 * i + 2)
                    self.tree[i] = temp
                    j = 2 * i + 2
                    while self.tree[j] != temp:
                        if temp < self.tree[j]:
                            j = 2 * j + 1
                        else:
                            j = 2 * j + 2
                    self.tree[j] = None
                    return True
            elif self.tree[i] > data:
                i = 2 * i + 1
            else:
                i = 2 * i + 2
        return False
```

7a.py - D:/DSA-25054/Week-7/7a.py (3.13.7)

File Edit Format Run Options Window Help

```
    def inorder(self, i):
        if i >= 100 or self.tree[i] is None:
            return
        self.inorder(2 * i + 1)
        print(self.tree[i], end=" ")
        self.inorder(2 * i + 2)

    def preorder(self, i):
        if i >= 100 or self.tree[i] is None:
            return
        print(self.tree[i], end=" ")
        self.preorder(2 * i + 1)
        self.preorder(2 * i + 2)

    def postorder(self, i):
        if i >= 100 or self.tree[i] is None:
            return
        self.postorder(2 * i + 1)
        self.postorder(2 * i + 2)
        print(self.tree[i], end=" ")

    def count(self, i):
        if i >= 100 or self.tree[i] is None:
            return 0
        return 1 + self.count(2 * i + 1) + self.count(2 * i + 2)

    def height(self, i):
        if i >= 100 or self.tree[i] is None:
            return 0
        left = self.height(2 * i + 1)
        right = self.height(2 * i + 2)
        return 1 + max(left, right)

# Main
bst = BST()
bst.insert(50)
bst.insert(30)
bst.insert(70)
bst.insert(20)
bst.insert(40)
bst.insert(60)
bst.insert(80)

print("Inorder:")
bst.inorder(0)

print("\nPreorder:")
bst.preorder(0)

print("\nPostorder:")
bst.postorder(0)

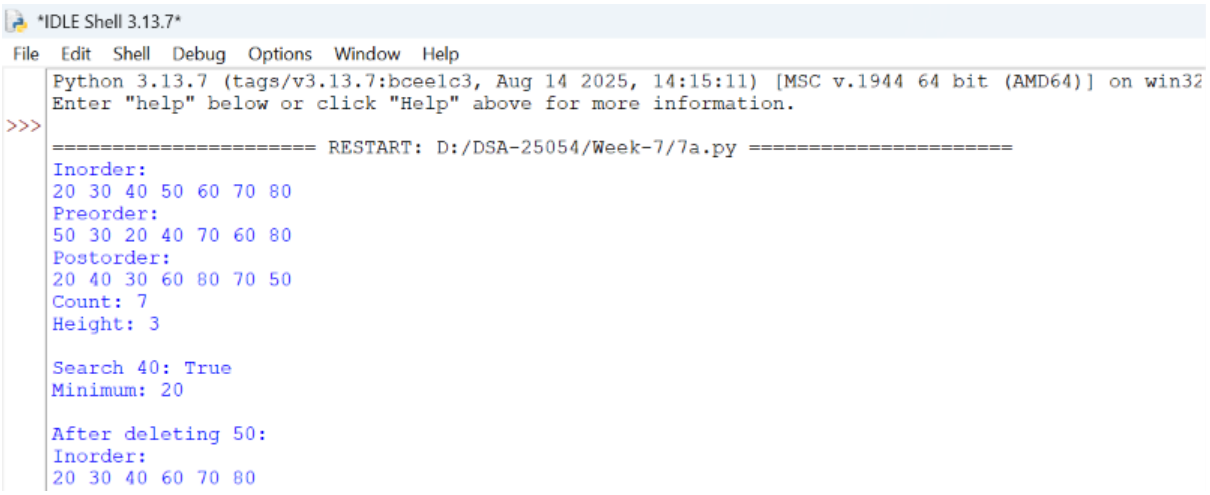
print("\nCount:", bst.count(0))
print("Height:", bst.height(0))

print("\nSearch 40:", bst.search(40))
print("Minimum:", bst.min(0))

bst.delete(50)

print("\nAfter deleting 50:")
print("Inorder:")
bst.inorder(0)
```

Output:



Explanation:

Binary Search Tree (BST) Using Array

Structure:

- Uses a Python list self.tree to store the elements.
- The root element is stored at **index 0**.
- For a node at index i:
    - **Left child** → 2*i + 1
    - **Right child** → 2*i + 2
- An empty position is represented using None.
- BST follows the rule:
    - Left subtree contains **smaller** elements.
    - Right subtree contains **greater** elements.

Operations:

1. Insert

- Start from index 0 (root).
- Compare the new element with the current element.
- If the element is smaller → move to 2*i + 1.
- If the element is greater → move to 2*i + 2.
- Continue until an empty position (None) is found.
- Insert the element at that position.

2. Search

- Start from the root.
- Compare the required element with the current element.
- If equal → element is found.
- If smaller → move to the left child.
- If greater → move to the right child.
- If None is reached → element is not present.

3. Delete

There are three cases:

- **No child:** Set that position to None.
- **One child:** Replace the node with its child.
- **Two children:** Find the minimum element in the right subtree (inorder successor), replace the deleted element with it, and remove the original successor.

4. Minimum

- Start from the given node.
- Keep moving to the left child.
- The leftmost element is the minimum value.

5. Maximum

- Start from the given node.
- Keep moving to the right child.
- The rightmost element is the maximum value.

6. Inorder Traversal

- Traverse **Left → Root → Right**.
- Inorder traversal of a BST gives elements in **sorted order**.

7. Preorder Traversal

- Traverse **Root → Left → Right**.

8. Postorder Traversal

- Traverse **Left → Right → Root**.

9. Count

- Counts the total number of nodes in the BST.
- Formula:
  Count = 1 + left subtree count + right subtree count

10. Height

- Finds the longest path from the root to a leaf.
- Formula:
  Height = 1 + max(left height, right height)

Important Points:

- Root is at index 0.
- Left child → 2*i + 1.
- Right child → 2*i + 2.
- None represents an empty position.
- Inorder traversal gives **sorted elements**.
- BST provides efficient searching when the tree is balanced.

Error Table:

S.No.	Errors Observed	Rectification
1.	Inserting into an occupied position without comparing the value.	Compare data with self.tree[i]. If data is smaller, move to 2*i + 1; otherwise move to 2*i + 2.
2.	Searching beyond the array limit.	Check the index condition: while i< 100 and self.tree[i] is not None.
3.	Searching for a value that does not exist.	If self.tree[i] is None, return False.
4.	Incorrect child index calculation.	Use left child = 2*i + 1 and right child = 2*i + 2.

Program: Using Linked List

7b.py - D:/DSA-25054/Week-7/7b.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, data):
        new = Node(data)
        if self.root is None:
            self.root = new
            return

        temp = self.root
        while True:
            if data < temp.data:
                if temp.left is None:
                    temp.left = new
                    break
                else:
                    temp = temp.left
            else:
                if temp.right is None:
                    temp.right = new
                    break
                else:
                    temp = temp.right

    def search(self, data):
        temp = self.root
        while temp is not None:
            if temp.data == data:
                return True
            if data < temp.data:
                temp = temp.left
            else:
                temp = temp.right
        return False
```

7b.py - D:/DSA-25054/Week-7/7b.py (3.13.7)

File Edit Format Run Options Window Help

```
        return False

    def min(self, root):
        temp = root
        while temp.left is not None:
            temp = temp.left
        return temp.data

    def max(self, root):
        temp = root
        while temp.right is not None:
            temp = temp.right
        return temp.data

    def delete(self, root, data):
        if root is None:
            return None

        if data < root.data:
            root.left = self.delete(root.left, data)
        elif data > root.data:
            root.right = self.delete(root.right, data)
        else:
            # No child
            if root.left is None and root.right is None:
                return None
            # Only right child
            elif root.left is None:
                return root.right
            # Only left child
            elif root.right is None:
                return root.left
            # Two children
            else:
                temp = self.min(root.right)
                root.data = temp
                root.right = self.delete(root.right, temp)
        return root
```

7b.py - D:/DSA-25054/Week-7/7b.py (3.13.7)

File Edit Format Run Options Window Help

```
                root.right = self.delete(root.right, temp)
        return root

    def inorder(self, root):
        if root is None:
            return
        self.inorder(root.left)
        print(root.data, end=" ")
        self.inorder(root.right)

    def preorder(self, root):
        if root is None:
            return
        print(root.data, end=" ")
        self.preorder(root.left)
        self.preorder(root.right)

    def postorder(self, root):
        if root is None:
            return
        self.postorder(root.left)
        self.postorder(root.right)
        print(root.data, end=" ")

    def count(self, root):
        if root is None:
            return 0
        return 1 + self.count(root.left) + self.count(root.right)

    def height(self, root):
        if root is None:
            return 0
        left = self.height(root.left)
        right = self.height(root.right)
        return 1 + max(left, right)
```

```
# Main
bst = BST()
bst.insert(50)
bst.insert(30)
bst.insert(70)
bst.insert(20)
bst.insert(40)
bst.insert(60)
bst.insert(80)

print("Inorder:")
bst.inorder(bst.root)

print("\nPreorder:")
bst.preorder(bst.root)

print("\nPostorder:")
bst.postorder(bst.root)

print("\nCount:", bst.count(bst.root))
print("Height:", bst.height(bst.root))

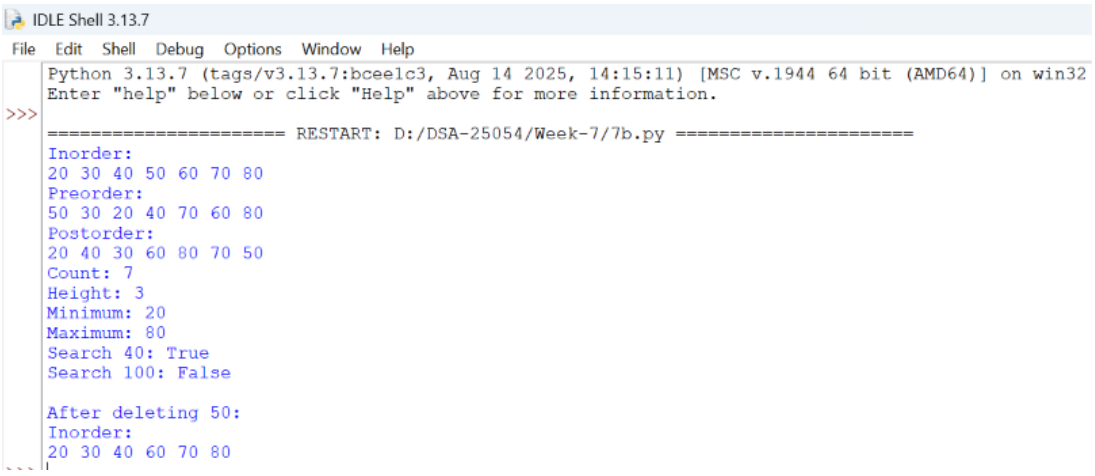
print("Minimum:", bst.min(bst.root))
print("Maximum:", bst.max(bst.root))

print("Search 40:", bst.search(40))
print("Search 100:", bst.search(100))

bst.root = bst.delete(bst.root, 50)

print("\nAfter deleting 50:")
print("Inorder:")
bst.inorder(bst.root)
```

Output:



Explanation:

Binary Search Tree (BST) Using Linked List

Structure:

- Each element is represented using a **Node**.
- Each Node contains:
  - data → stores the value.
  - left → points to the left child.
  - right → points to the right child.
- self.root points to the **root node** of the BST.
- Initially, root = None, which means the tree is empty.
- BST follows the rule:
  - Left subtree contains **smaller** elements.
  - Right subtree contains **greater** elements.

Operations:

1. Insert

- Create a new node.
- If the tree is empty, make the new node the root.
- Otherwise, start from the root.
- If the new element is smaller → move to the left.
- If the new element is greater → move to the right.
- Continue until an empty position is found.
- Link the new node at that position.

2. Search

- Start from the root.

- Compare the required element with the current node.
- If equal → element is found.
- If smaller → move to the left child.
- If greater → move to the right child.
- If None is reached → element is not present.

3. Delete

There are three cases:

- **No child:** Remove the node.
- **One child:** Replace the node with its child.
- **Two children:** Find the minimum element in the right subtree (inorder successor), copy its value into the node, and delete the original successor node.

4. Minimum

- Start from the given node.
- Keep moving to the left child.
- The leftmost node contains the minimum value.

5. Maximum

- Start from the given node.
- Keep moving to the right child.
- The rightmost node contains the maximum value.

6. Inorder Traversal

- Traverse **Left → Root → Right**.
- Gives BST elements in **sorted order**.

7. Preorder Traversal

- Traverse **Root → Left → Right**.

8. Postorder Traversal

- Traverse **Left → Right → Root**.

9. Count

- Counts the total number of nodes.
- Formula:
  Count = 1 + left subtree count + right subtree count

10. Height

- Finds the longest path from the root to a leaf.
- Formula:
  Height = 1 + max(left height, right height)

Important Points:

- root points to the first node.
- left stores the reference to the left child.
- right stores the reference to the right child.
- None represents an empty child.
- Inorder traversal gives **sorted elements**.
- Linked-list representation uses **dynamic memory**.

Error Table:

S.No.	Errors Observed	Rectification
1.	Inserting the first node without setting root.	If self.root is None, set self.root = new and return.
2.	Searching for a value that does not exist.	Continue while temp is not None; if temp becomes None, return False.
3.	Incorrect movement while searching.	If data <temp.data, move to temp.left; otherwise move to temp.right.
4.	Deleting a node with two children incorrectly.	Find the minimum value in the right subtree, replace the node data, and delete the successor.

WEEK-8

8A)**Aim:** Write a python program to implement Binary tree with its operations using array and linked list.

Program: Using Array

8a.py - D:/DSA-25054/Week-8/8a.py (3.13.7)

File Edit Format Run Options Window Help

```
class BinaryTreeArray:
    def __init__(self, size=50):
        self.size = size
        self.tree = [None] * size

    def insert_root(self, key):
        self.tree[0] = key

    def set_left(self, key, parent):
        child = 2 * parent + 1
        if parent < self.size and self.tree[parent] is not None:
            if child < self.size:
                self.tree[child] = key
            else:
                print("Index out of range")
        else:
            print("Parent does not exist")

    def set_right(self, key, parent):
        child = 2 * parent + 2
        if parent < self.size and self.tree[parent] is not None:
            if child < self.size:
                self.tree[child] = key
            else:
                print("Index out of range")
        else:
            print("Parent does not exist")

    def preorder(self, index=0):
        if index < self.size and self.tree[index] is not None:
            print(self.tree[index], end=" ")
            self.preorder(2 * index + 1)
            self.preorder(2 * index + 2)

    def inorder(self, index=0):
        if index < self.size and self.tree[index] is not None:
            self.inorder(2 * index + 1)
            print(self.tree[index], end=" ")
            self.inorder(2 * index + 2)
```

8a.py - D:/DSA-25054/Week-8/8a.py (3.13.7)

File Edit Format Run Options Window Help

```
    def preorder(self, index=0):
        if index < self.size and self.tree[index] is not None:
            print(self.tree[index], end=" ")
            self.preorder(2 * index + 1)
            self.preorder(2 * index + 2)

    def inorder(self, index=0):
        if index < self.size and self.tree[index] is not None:
            self.inorder(2 * index + 1)
            print(self.tree[index], end=" ")
            self.inorder(2 * index + 2)

    def postorder(self, index=0):
        if index < self.size and self.tree[index] is not None:
            self.postorder(2 * index + 1)
            self.postorder(2 * index + 2)
            print(self.tree[index], end=" ")

    def levelorder(self):
        for val in self.tree:
            if val is not None:
                print(val, end=" ")

    def display(self):
        for i in range(self.size):
            if self.tree[i] is not None:
                print(f"Index {i}: {self.tree[i]}")

# Driver code
bt = BinaryTreeArray(15)
bt.insert_root("A")
bt.set_left("B", 0)
bt.set_right("C", 0)
bt.set_left("D", 1)
bt.set_right("E", 1)

print("Inorder traversal:")
bt.inorder(0)
print("\nPreorder traversal:")
bt.preorder(0)
print("\nPostorder traversal:")
bt.postorder(0)
print("\nLevelorder traversal:")
bt.levelorder()
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: D:/DSA-25054/Week-8/8a.py ==========
Inorder traversal:
D B E A C
Preorder traversal:
A B D E C
Postorder traversal:
D E B C A
Levelorder traversal:
A B C D E
```

Explanation:

- The binarytree class is used to implement a binary tree using an array. The tree list is initialized with None values according to the given size.
- The insert_root() function stores the root node at index 0 because the root is always the first element in the array representation.

- The set_left() function is used to insert the left child of a node. For a node at index i, its left child is stored at index **2*i + 1**.
- The set_right() function is used to insert the right child of a node. For a node at index i, its right child is stored at index **2*i + 2**.
- The if child <self.size condition checks whether the calculated child index is within the available array size. If it exceeds the size, an index-out-of-range message is displayed.
- The preorder() function visits the current node first, followed by its left subtree and right subtree. The child indexes are calculated using 2*index + 1 and 2*index + 2.
- The inorder() function first visits the left subtree, then the current node and finally the right subtree.
- The postorder() function first visits the left subtree, then the right subtree and finally the current node.
- The levelorder() function visits the elements from the beginning to the end of the array. Since the array stores the binary tree level by level, this directly gives the level-order traversal.
- The display() function uses a loop to display every index of the array along with the value stored at that index.
- The condition self.tree[index] is None is used during traversal to stop when an empty position is reached.

Error Table:

S.No.	Errors Observed	Rectification
1	Not setting left and right references to None while creating a node.	Initialize both left and right as None in the Node constructor.
2	Calling traversal functions without passing the current node correctly.	Pass the current node during recursive calls.
3	Forgetting the base condition when the current node is None.	Use a condition such as if root is None: return.
4	Using the wrong traversal order.	Preorder: Root-Left-Right, Inorder: Left-Root-Right, Postorder: Left-Right-Root.

Program: Using Linked List

8b.py - D:\DSA-25054\Week-8\8b.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self):
        self.root = None

    # Preorder Traversal: Root -> Left -> Right
    def preorder(self, root):
        if root:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    # Inorder Traversal: Left -> Root -> Right
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    # Postorder Traversal: Left -> Right -> Root
    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")

    # Level Order Traversal (BFS)
    def levelorder(self, root):
        if root is None:
            return
        queue = [root]
        while len(queue) > 0:
            current = queue.pop(0)
            print(current.data, end=" ")
            if current.left is not None:
                queue.append(current.left)
            if current.right is not None:
                queue.append(current.right)
```

```
tree = BinaryTree()

tree.root = Node("A")
tree.root.left = Node("B")
tree.root.right = Node("C")
tree.root.left.left = Node("D")
tree.root.left.right = Node("E")

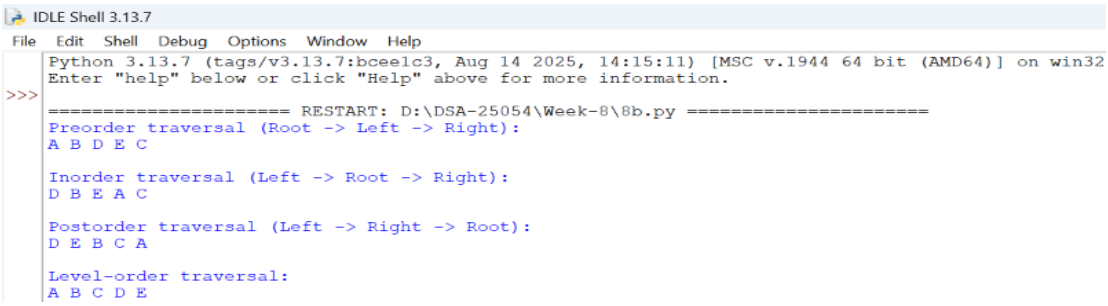
print("Preorder traversal (Root -> Left -> Right):")
tree.preorder(tree.root)

print("\n\nInorder traversal (Left -> Root -> Right):")
tree.inorder(tree.root)

print("\n\nPostorder traversal (Left -> Right -> Root):")
tree.postorder(tree.root)

print("\n\nLevel-order traversal:")
tree.levelorder(tree.root)
```

Output:



Explanation:

- The Node class is used to create each node of the binary tree. Each node contains three parts: data, left, and right. The data stores the value, while left and right store references to the left and right child nodes.
- The __init__() method initializes the node with the given data. Initially, both left and right are set to None because the node does not have any children.
- The root represents the first node of the binary tree. New nodes are connected to the root using the left and right references.
- The preorder() function performs preorder traversal. It first visits the root node, then recursively visits the left subtree and finally the right subtree. The order followed is **Root → Left → Right**.
- The inorder() function performs inorder traversal. It first visits the left subtree, then the root node and finally the right subtree. The order followed is **Left → Root → Right**.
- The postorder() function performs postorder traversal. It first visits the left subtree, then the right subtree and finally the root node. The order followed is **Left → Right → Root**.
- The condition if root: checks whether the current node exists. If the node is None, the recursive function stops for that particular branch.

Error Table:

S.No.	Errors Observed	Rectification
1	Not setting left and right references to None while creating a node.	Initialize both left and right as None in the Node constructor.
2	Calling traversal functions without passing the current node correctly.	Pass the current node during recursive calls.
3	Forgetting the base condition when the current node is None.	Use a condition such as if root is None: return.
4	Using the wrong traversal order.	Preorder: Root-Left-Right, Inorder: Left-Root-Right, Postorder: Left-Right-Root.

WEEK-9

9A)Aim: Write a python code for AVL tree using array and linked list with all its operations.

Program: Using Linked List

9a.py - D:/DSA-25054/Week-9/9a.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1

class AVLTree:
    def height(self, node):
        if not node:
            return 0
        return node.height

    def get_balance(self, node):
        if not node:
            return 0
        return self.height(node.left) - self.height(node.right)

    def right_rotate(self, y):
        x = y.left
        T2 = x.right

        x.right = y
        y.left = T2

        y.height = 1 + max(self.height(y.left), self.height(y.right))
        x.height = 1 + max(self.height(x.left), self.height(x.right))

        return x

    def left_rotate(self, x):
        y = x.right
        T2 = y.left

        y.left = x
        x.right = T2

        x.height = 1 + max(self.height(x.left), self.height(x.right))
        y.height = 1 + max(self.height(y.left), self.height(y.right))

        return y
```

9a.py - D:/DSA-25054/Week-9/9a.py (3.13.7)

File Edit Format Run Options Window Help

```
        return y

    def insert(self, root, key):
        if not root:
            return Node(key)
        elif key < root.data:
            root.left = self.insert(root.left, key)
        elif key > root.data:
            root.right = self.insert(root.right, key)
        else:
            return root

        root.height = 1 + max(self.height(root.left), self.height(root.right))
        balance = self.get_balance(root)

        # LL Case
        if balance > 1 and key < root.left.data:
            return self.right_rotate(root)

        # RR Case
        if balance < -1 and key > root.right.data:
            return self.left_rotate(root)

        # LR Case
        if balance > 1 and key > root.left.data:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RL Case
        if balance < -1 and key < root.right.data:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    def min_value_node(self, node):
        current = node
        while current.left is not None:
            current = current.left
        return current
```

9a.py - D:/DSA-25054/Week-9/9a.py (3.13.7)

File Edit Format Run Options Window Help

```
    def delete(self, root, key):
        if not root:
            return root

        if key < root.data:
            root.left = self.delete(root.left, key)
        elif key > root.data:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = self.min_value_node(root.right)
            root.data = temp.data
            root.right = self.delete(root.right, temp.data)

        if root is None:
            return root

        root.height = 1 + max(self.height(root.left), self.height(root.right))
        balance = self.get_balance(root)

        # LL Case
        if balance > 1 and self.get_balance(root.left) >= 0:
            return self.right_rotate(root)

        # LR Case
        if balance > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RR Case
        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)

        # RL Case
        if balance < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root
```

9a.py - D:/DSA-25054/Week-9/9a.py (3.13.7)

File Edit Format Run Options Window Help

```
    def search(self, root, key):
        if root is None or root.data == key:
            return root is not None
        if key < root.data:
            return self.search(root.left, key)
        return self.search(root.right, key)

    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")

    def display(self, root, space=0):
        if root is None:
            return
        space += 5
        self.display(root.right, space)
        print(" " * (space - 5) + str(root.data))
        self.display(root.left, space)
```

9a.py - D:/DSA-25054/Week-9/9a.py (3.13.7)

File Edit Format Run Options Window Help

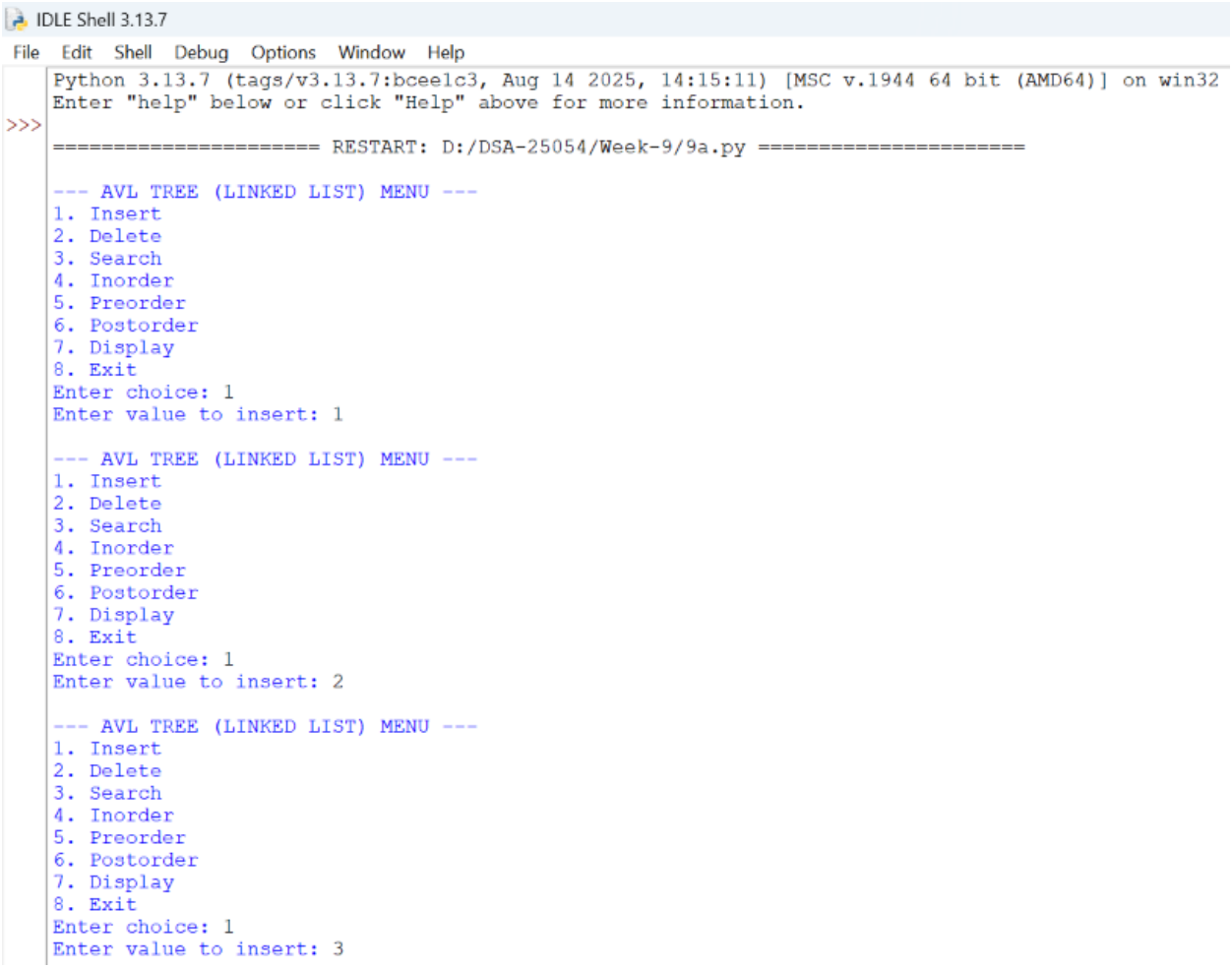
```
# Menu-driven Program
avl = AVLTree()
root = None

while True:
    print("\n--- AVL TREE (LINKED LIST) MENU ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder")
    print("5. Preorder")
    print("6. Postorder")
    print("7. Display")
    print("8. Exit")

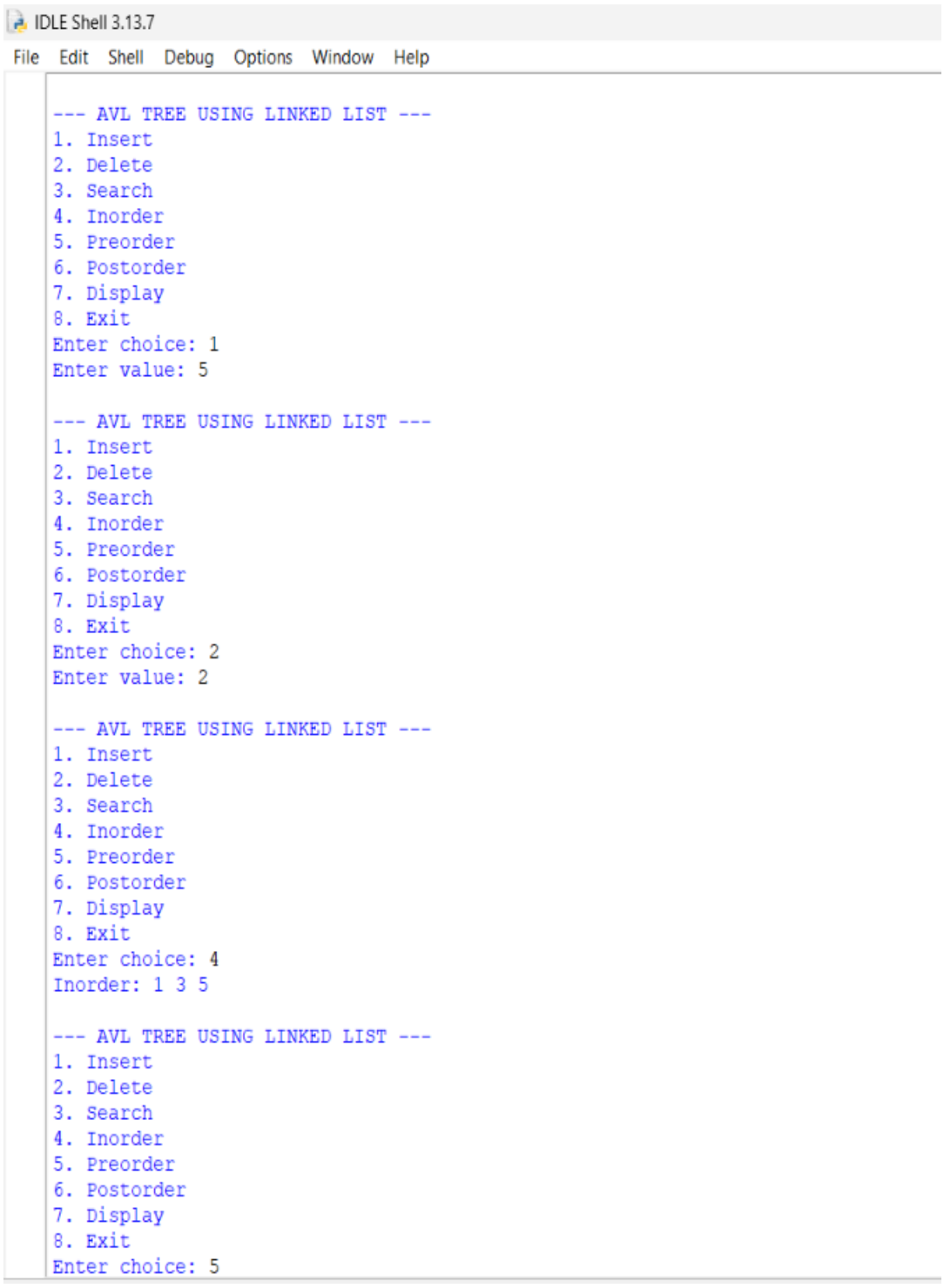
    choice = int(input("Enter choice: "))

    if choice == 1:
        val = int(input("Enter value to insert: "))
        root = avl.insert(root, val)
    elif choice == 2:
        val = int(input("Enter value to delete: "))
        root = avl.delete(root, val)
    elif choice == 3:
        val = int(input("Enter value to search: "))
        print("Found" if avl.search(root, val) else "Not Found")
    elif choice == 4:
        print("Inorder traversal: ", end="")
        avl.inorder(root)
        print()
    elif choice == 5:
        print("Preorder traversal: ", end="")
        avl.preorder(root)
        print()
    elif choice == 6:
        print("Postorder traversal: ", end="")
        avl.postorder(root)
        print()
    elif choice == 7:
        avl.display(root)
    elif choice == 8:
        print("Exiting...")
        break
    else:
        print("Invalid choice")
```

Output:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: D:/DSA-25054/Week-9/9a.py ==========
--- AVL TREE (LINKED LIST) MENU ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 1
Enter value to insert: 1
--- AVL TREE (LINKED LIST) MENU ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 1
Enter value to insert: 2
--- AVL TREE (LINKED LIST) MENU ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 1
Enter value to insert: 3


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
--- AVL TREE (LINKED LIST) MENU ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 1
Enter value to insert: 5
--- AVL TREE (LINKED LIST) MENU ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 2
Enter value to delete: 2
--- AVL TREE (LINKED LIST) MENU ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 4
Inorder traversal: 1 3 5


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 1
Enter value: 5
--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 2
Enter value: 2
--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 4
Inorder: 1 3 5
--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 5

```
8. Exit
Enter choice: 5
Preorder: 3 1 5

--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 6
Postorder: 1 5 3

--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 7

AVL Tree:
    5
3
    1

--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 3
Enter value: 12
Element not found

--- AVL TREE USING LINKED LIST ---
1. Insert
```

```
--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 3
Enter value: 12
Element not found

--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 8
>>>
```

Error Table:

S.No.	Errors Observed	Rectification
1	AttributeError: NoneType	The code tries to access a property of an empty node. Check whether the node is None before accessing its data or children.
2	Incorrect rotation	The left/right child references are changed incorrectly. Verify the temporary node and child links during rotation.
3	Wrong height or balance factor	The height is not updated after insertion, deletion, or rotation. Call the height update function at the correct place.
4	Element not found during deletion/search	The value does not exist in the tree or the search path is incorrect. Follow BST comparison rules correctly.

Program: Using Arrays

9b.py - D:\DSA-25054\Week-9\9b.py (3.13.7)

File Edit Format Run Options Window Help

```
class AVLTreeArray:
    def __init__(self, size=64):
        self.size = size
        self.tree = [None] * size

    def height(self, i=0):
        if i >= self.size or self.tree[i] is None:
            return 0
        left_h = self.height(2 * i + 1)
        right_h = self.height(2 * i + 2)
        return 1 + max(left_h, right_h)

    def balance_factor(self, i):
        if i >= self.size or self.tree[i] is None:
            return 0
        return self.height(2 * i + 1) - self.height(2 * i + 2)

    def get_subtree(self, i):
        """Extracts (relative_index, value) tuples for a subtree."""
        if i >= self.size or self.tree[i] is None:
            return []
        nodes = [(0, self.tree[i])]
        queue = [(i, 0)]
        while queue:
            curr_idx, rel_idx = queue.pop(0)
            left_child = 2 * curr_idx + 1
            right_child = 2 * curr_idx + 2
            if left_child < self.size and self.tree[left_child] is not None:
                queue.append((left_child, 2 * rel_idx + 1))
                nodes.append((2 * rel_idx + 1, self.tree[left_child]))
            if right_child < self.size and self.tree[right_child] is not None:
                queue.append((right_child, 2 * rel_idx + 2))
                nodes.append((2 * rel_idx + 2, self.tree[right_child]))
        return nodes

    def clear_subtree(self, i):
        if i >= self.size or self.tree[i] is None:
            return
        self.clear_subtree(2 * i + 1)
        self.clear_subtree(2 * i + 2)
        self.tree[i] = None
```

9b.py - D:\DSA-25054\Week-9\9b.py (3.13.7)

File Edit Format Run Options Window Help

```
        self.tree[i] = None

    def right_rotate(self, root_idx):
        left_idx = 2 * root_idx + 1
        new_root = self.tree[left_idx]
        t2_nodes = self.get_subtree(2 * left_idx + 2)
        right_nodes = self.get_subtree(2 * root_idx + 2)
        t1_nodes = self.get_subtree(2 * left_idx + 1)

        self.clear_subtree(root_idx)
        self.tree[root_idx] = new_root

        # Place new right child (old root)
        right_child = 2 * root_idx + 2
        if right_child < self.size:
            self.tree[right_child] = self.tree[root_idx]  # Old root
        # Reconnect T1, T2, and previous right subtree
        self._rebuild_from_list(self.get_inorder_list(0))

    def left_rotate(self, root_idx):
        self._rebuild_from_list(self.get_inorder_list(0))

    def _rebuild_from_list(self, sorted_vals, i=0):
        if not sorted_vals or i >= self.size:
            return
        mid = len(sorted_vals) // 2
        self.tree[i] = sorted_vals[mid]
        self._rebuild_from_list(sorted_vals[:mid], 2 * i + 1)
        self._rebuild_from_list(sorted_vals[mid + 1:], 2 * i + 2)
```

9b.py - D:\DSA-25054\Week-9\9b.py (3.13.7)

File Edit Format Run Options Window Help

```
    def insert(self, key):
        i = 0
        while i < self.size:
            if self.tree[i] is None:
                self.tree[i] = key
                self.rebalance()
                print(f"{key} inserted into AVL tree")
                return
            elif key < self.tree[i]:
                i = 2 * i + 1
            elif key > self.tree[i]:
                i = 2 * i + 2
            else:
                print("Duplicate keys not allowed")
                return
        print("Tree full / Index out of range")

    def rebalance(self):
        vals = self.get_inorder_list(0)
        self.tree = [None] * self.size
        self._rebuild_from_list(vals, 0)

    def search(self, key):
        i = 0
        while i < self.size and self.tree[i] is not None:
            if self.tree[i] == key:
                return True
            elif key < self.tree[i]:
                i = 2 * i + 1
            else:
                i = 2 * i + 2
        return False

    def delete(self, key):
        if not self.search(key):
            print("Element not found")
            return
        vals = self.get_inorder_list(0)
        vals.remove(key)
        self.tree = [None] * self.size
        self._rebuild_from_list(vals, 0)
        print(f"Element {key} deleted")
```

9b.py - D:\DSA-25054\Week-9\9b.py (3.13.7)

File Edit Format Run Options Window Help

```
        self._rebuild_from_list(vals, 0)
        print(f"Element {key} deleted")

    def get_inorder_list(self, i=0):
        if i >= self.size or self.tree[i] is None:
            return []
        return self.get_inorder_list(2 * i + 1) + [self.tree[i]] + self.get_inorder_list(2 * i + 2)

    def inorder(self, i=0):
        if i < self.size and self.tree[i] is not None:
            self.inorder(2 * i + 1)
            print(self.tree[i], end=" ")
            self.inorder(2 * i + 2)

    def preorder(self, i=0):
        if i < self.size and self.tree[i] is not None:
            print(self.tree[i], end=" ")
            self.preorder(2 * i + 1)
            self.preorder(2 * i + 2)

    def postorder(self, i=0):
        if i < self.size and self.tree[i] is not None:
            self.postorder(2 * i + 1)
            self.postorder(2 * i + 2)
            print(self.tree[i], end=" ")

    def display(self):
        print(f"{'Index':<8}{'Data':<8}{'Left':<8}{'Right':<8}{'Height':<8}")
        for i in range(self.size):
            if self.tree[i] is not None:
                left = 2 * i + 1 if (2 * i + 1 < self.size and self.tree[2 * i + 1] is not None) else -1
                right = 2 * i + 2 if (2 * i + 2 < self.size and self.tree[2 * i + 2] is not None) else -1
                h = self.height(i)
                print(f"{i:<8}{self.tree[i]:<8}{left:<8}{right:<8}{h:<8}")
```

9b.py - D:\DSA-25054\Week-9\9b.py (3.13.7)

File Edit Format Run Options Window Help

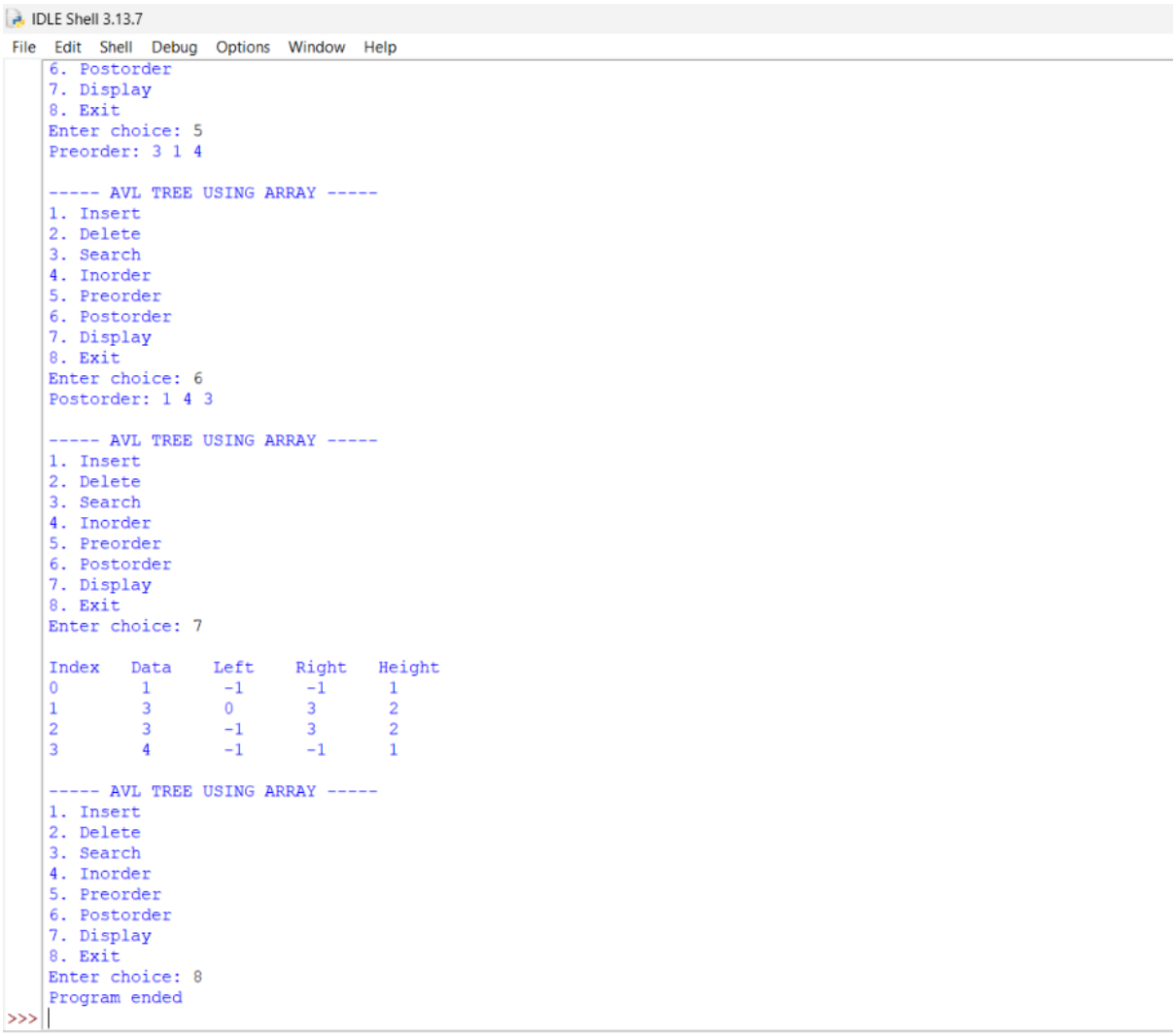
```
                right = 2 * i + 2 if (2 * i + 2 < self.size and self.tree[2 * i + 2] is not None) else -1
                h = self.height(i)
                print(f"{i:<8}{self.tree[i]:<8}{left:<8}{right:<8}{h:<8}")

# Menu-driven Driver Program
avl = AVLTreeArray(64)
while True:
    print("\n--- AVL TREE (ARRAY) MENU ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder")
    print("5. Preorder")
    print("6. Postorder")
    print("7. Display")
    print("8. Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        val = int(input("Enter value to insert: "))
        avl.insert(val)
    elif choice == 2:
        val = int(input("Enter value to delete: "))
        avl.delete(val)
    elif choice == 3:
        val = int(input("Enter value to search: "))
        print("Found" if avl.search(val) else "Not Found")
    elif choice == 4:
        print("Inorder: ", end="")
        avl.inorder(0)
        print()
    elif choice == 5:
        print("Preorder: ", end="")
        avl.preorder(0)
        print()
    elif choice == 6:
        print("Postorder: ", end="")
        avl.postorder(0)
        print()
    elif choice == 7:
        avl.display()
    elif choice == 8:
        print("Exiting...")
        break
    else:
        print("Invalid choice")
```

Output:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: D:/DSA-25054/Week-9/9b.py ==========
--- AVL TREE (ARRAY) MENU ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 1
Enter value to insert: 1
--- AVL TREE (ARRAY) MENU ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 1
Enter value to insert: 2
--- AVL TREE (ARRAY) MENU ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 1
Enter value to insert: 3


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
6. Postorder
7. Display
8. Exit
Enter choice: 1
Enter value: 4
Element inserted
----- AVL TREE USING ARRAY -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 2
Enter value: 2
Element deleted
----- AVL TREE USING ARRAY -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 4
Inorder: 1 3 4
----- AVL TREE USING ARRAY -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 5
Preorder: 3 1 4
----- AVL TREE USING ARRAY -----
1. Insert
2. Delete



Explanation:

- An **AVL tree** is a self-balancing Binary Search Tree (BST).
- In an AVL tree, the difference between the height of the left and right subtrees is called the **Balance Factor**.
- The Balance Factor of every node must be **-1, 0, or +1**.
- In the array implementation, the tree nodes are stored inside a fixed-size array.
- For a node at index i:
    - Left child is stored at 2*i + 1.

    - Right child is stored at 2*i + 2.
- None is used to represent an empty position in the array.

Main Functions

- **__init__()**
    - Creates the array for storing the tree.
    - The size of the array is given by the user/program.
    - Initially, all positions contain None.
- **height()**
    - Calculates the height of a node.
    - It recursively checks the left and right subtrees.
    - The greater height is selected and 1 is added.
- **balance_factor()**
    - Calculates the balance factor of a node.
    - It is calculated as:
        - Height of Left Subtree - Height of Right Subtree
    - A value greater than 1 indicates that the tree is left-heavy.
    - A value less than -1 indicates that the tree is right-heavy.
- **insert()**
    - Adds a new element to the AVL tree.
    - The element is placed according to Binary Search Tree rules.
    - Smaller values are placed in the left subtree.
    - Larger values are placed in the right subtree.
    - After insertion, the tree is checked and rotations are performed if required.

- **search()**
  - Searches for a particular element.
  - If the element is found, it returns True.
  - If the element does not exist, it returns False.
- **delete()**
  - Removes an element from the tree.
  - It handles nodes with no child, one child, or two children.
  - After deletion, the tree is rebalanced.
- **find_min()**
  - Finds the smallest element in a subtree.
  - It repeatedly moves to the left child.
  - It is mainly used when deleting a node having two children.
- **rotate_left()**
  - Performs a left rotation when the tree becomes right-heavy.
  - It helps restore the AVL property.
- **rotate_right()**
  - Performs a right rotation when the tree becomes left-heavy.
  - It helps restore the AVL property.
- **balance_tree()**
  - Checks the balance factor of the nodes.
  - If the tree becomes unbalanced, suitable rotations are performed.
- **inorder()**
  - Traverses the tree in:
    - Left → Root → Right

- It displays the elements in sorted order.

- **preorder()**
  - Traverses the tree in:
    - Root → Left → Right
- **postorder()**
  - Traverses the tree in:
    - Left → Right → Root
- **display()**
  - Displays the elements along with their array indexes.
  - It helps to understand how the tree is stored in the array.

Main Program

- The main program provides a **menu-driven interface**.
- The user can choose:
  - Insert
  - Delete
  - Search
  - Inorder
  - Preorder
  - Postorder
  - Display
  - Exit
- This allows all major AVL tree operations to be performed interactively.

Error Table:

S.No.	Errors Observed	Rectification
1	Index out of range	The calculated child index exceeds the array size. Use a sufficiently large array and check the index before accessing it.
2	Tree becomes full	The fixed-size array has no empty position available. Increase the array size.
3	Incorrect child index	Using the wrong formula for child positions causes incorrect tree structure. Use left = 2*i+1 and right = 2*i+2.
4	Incorrect balancing after insertion/deletion	The balance factor or rotation is not handled correctly. Check the balance factor and perform the required rotation.

WEEK-10

10A)Aim: Write a Python program to implement hashing using separate chaining, linear probing, and quadratic probing.

Program:

10a.py - D:/DSA-25054/Week-10/10a.py (3.13.7)

File Edit Format Run Options Window Help

```
size = 10

# Separate Chaining
chain = [[] for _ in range(size)]

def separate_chaining(key):
    index = key % size
    chain[index].append(key)

# Linear Probing
linear = [None] * size

def linear_probing(key):
    index = key % size
    start = index

    while linear[index] is not None:
        index = (index + 1) % size

        if index == start:
            print("Hash table is full")
            return

    linear[index] = key

# Quadratic Probing
quadratic = [None] * size

def quadratic_probing(key):
    index = key % size
    i = 0

    while i < size:
        new_index = (index + i * i) % size

        if quadratic[new_index] is None:
            quadratic[new_index] = key
            return

        i += 1

    print("Cannot insert key:", key)
```

```
# Input keys
keys = list(map(int, input("Enter keys separated by spaces: ").split()))

for key in keys:
    separate_chaining(key)
    linear_probing(key)
    quadratic_probing(key)

print("\nSeparate Chaining:")
for i in range(size):
    print(i, ":", chain[i])

print("\nLinear Probing:")
for i in range(size):
    print(i, ":", linear[i])

print("\nQuadratic Probing:")
for i in range(size):
    print(i, ":", quadratic[i])
```

Output:

```
IDLE Shell 3.13.7
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: D:/DSA-25054/Week-10/10a.py ==========
Enter keys separated by spaces: 11 12 10 9

Separate Chaining:
0 : [10]
1 : [11]
2 : [12]
3 : []
4 : []
5 : []
6 : []
7 : []
8 : []
9 : [9]

Linear Probing:
0 : 10
1 : 11
2 : 12
3 : None
4 : None
5 : None
6 : None
7 : None
8 : None
9 : 9

Quadratic Probing:
0 : 10
1 : 11
2 : 12
3 : None
4 : None
5 : None
6 : None
7 : None
8 : None
9 : 9
```

Explanation:

Separate Chaining

- A hash table of size 10 is created.
- The hash function is key % size.
- If two keys have the same index, both are stored in the same list.
- This method handles collisions using lists.

Keys 12 and 22 both have index 2 because 12 % 10 = 2 and 22 % 10 = 2.

Linear Probing

- The key is first placed at the index calculated by the hash function.
- If that position is occupied, the next position is checked.
- The process continues until an empty position is found.

Quadratic Probing

- The key is placed at the calculated hash index if it is empty.
- If a collision occurs, positions are checked using the square of the attempt number.

Error Table:

S.No.	Error Observed	Rectification
1	Incorrect syntax in the program.	Correct the syntax.
2	Invalid input is entered.	Enter valid integer values.
3	Accessing an invalid index.	Use a valid index.
4	Two keys have the same hash index.	Use collision-handling techniques.
5	No empty position is available.	Increase the table size.